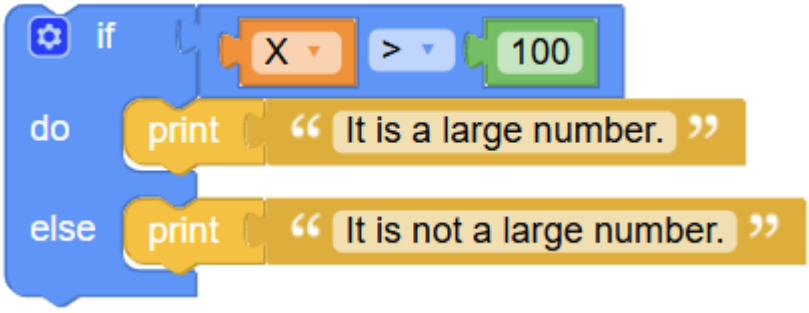
Basic Blocks

Logic

Logic blocks are generally used to control condition blocks and loop blocks.

Below is an example:



If the value of variable **x** is greater than 100, the condition becomes **true** and the text “It is a large number.” is printed.
If the value of **x** is not greater than 100, the condition becomes **false** and “It is not a large number.” is printed.

Boolean is a simple data type that has only two possible values: - **True** - **False**

Boolean values can be stored in variables or passed to functions, and they can be used in the same way as numbers, text, or list values.

Blocks

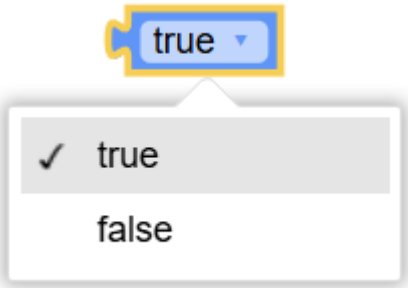
When a block requires a Boolean value as input, an empty input is generally interpreted as **false**.

You cannot directly connect a non-Boolean value to an input that expects a Boolean.
However, it is possible to store that value in a variable first and then pass it as input.

That said, this approach is not recommended, and its behavior may change in future versions.

Value

You can obtain a Boolean value by using a single block with a dropdown that allows you to select **True** or **False**:



JavaScript Code

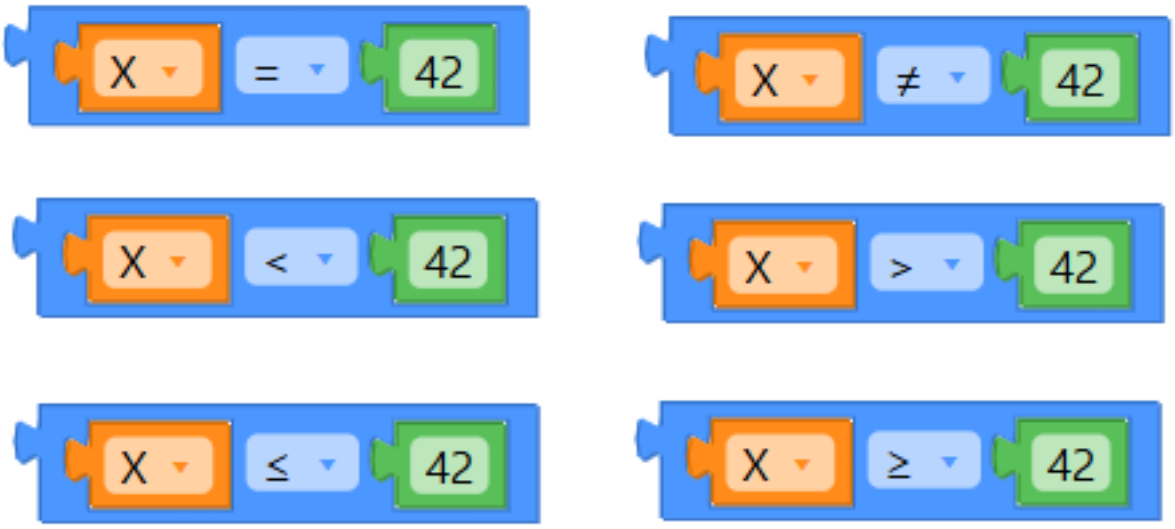
```
true    // true
false   // false
```

Python Code

```
True    # true
False   # false
```

Comparison

There are six comparison operators.
Each operator takes two inputs (such as numbers or text) and returns true or false depending on how the input values are compared.



The six comparison operators are as follows:

- equal to
- not equal to
- less than
- greater than
- less than or equal to
- greater than or equal to

JavaScript Code

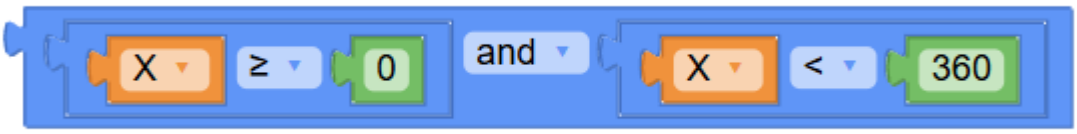
```
x == 42    // equal to
x != 42    // not equal to
x < 42     // less than
x > 42     // greater than
x <= 42    // less than or equal to
x >= 42    // greater than or equal to
```

Python Code

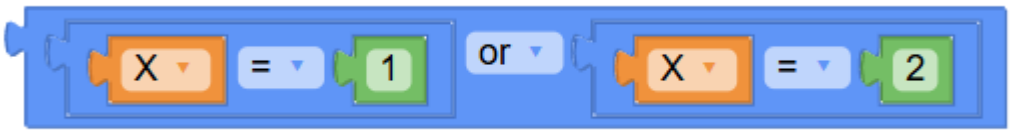
```
x == 42    # equal to
x != 42    # not equal to
x < 42     # less than
x > 42     # greater than
x <= 42    # less than or equal to
x >= 42    # greater than or equal to
```

Logical Operations

The **AND** block returns true only when both inputs are true.



The **OR** block returns true if at least one of the two inputs is true.



JavaScript Code

```
x >= 0 && x < 360    // AND
x == 1 || x == 2    // OR
```

Python Code

```
x >= 0 and x < 360   # AND
x == 1 or x == 2     # OR
```

NOT

The **NOT** block inverts a boolean input.
That is, it converts a boolean value to its opposite.

For example, the following results are produced:

False is returned.

As mentioned above, if no input is provided, the default value is considered **true**, so the block below returns **false**.

Leaving the input empty is not recommended.

JavaScript Code

```
!true          // or false
```

Python Code

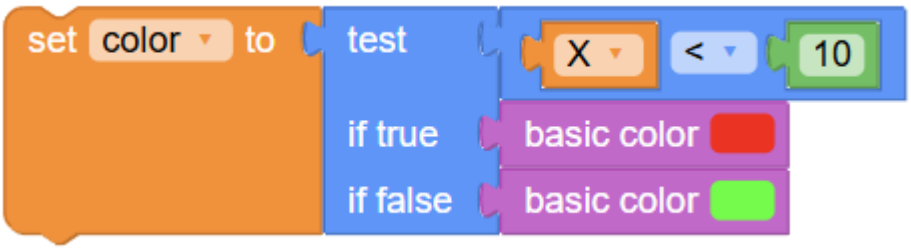
```
not True       # or False
```

Ternary Operator

The ternary block works like a simple if statement.
It takes three inputs.

The first input is a boolean condition to test.
The second input is the value to return when the condition is **true**.
The third input is the value to return when the condition is **false**.

In the example below, if the variable **x** is less than 10, the variable **color** is set to red; otherwise, the variable **color** is set to green.



The ternary block can always be replaced with a conditional (if) block.
The two examples below behave the same way.

JavaScript

```
x == 0 ? 'Game Over!' : 'Keep Playing'
```

Python Code

```
'Game Over!' if x == 0 else 'Keep Playing'
```

Conditional-Statement

Conditional statements play a crucial role in computer programming.

By using conditional statements, you can express sentences such as the following:

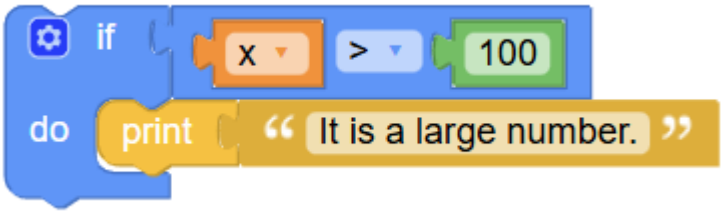
- If there is a path on the left, turn left.
- If the score is 100, print “Well done!”

Blocks

If

The simplest conditional statement is the **If** block.

It can be represented as shown below:



When this block is executed, it compares the value of variable **x** with 100.

If the value is greater than 100, “It is a large number.” is printed.

Otherwise, no code inside the block is executed.

Javascript Code

```
if (x > 100) {
    window.alert("It is a large number.");
```

```
}
```

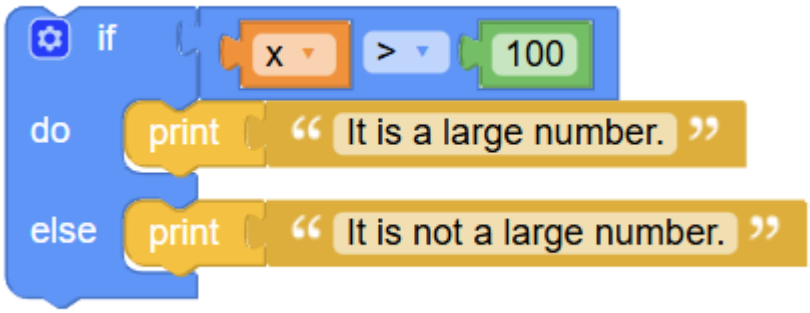
Python Code

```python
if x > 100:
    print("It is a large number.")
```

If–Else

You can also specify what to do when the condition is not true.

An example is shown below:



As in the previous block, if **x** is greater than 100, “It is a large number.” is printed.

Otherwise, “It is not a large number.” is printed.

The **If** block can have only one **Else** section, and no more than one is allowed.

Javascript Code

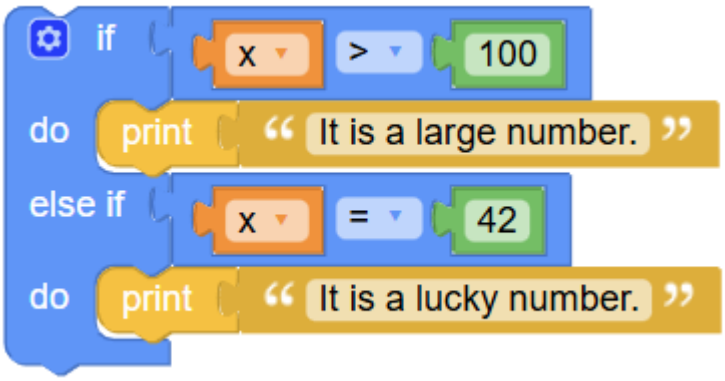
```javascript
if (x > 100) {
    window.alert('It is a large number.');
} else {
    window.alert('It is not a large number.');
}
```

Python Code

```python
if x > 100:
    print('It is a large number.')
else:
    print('It is not a large number.')
```

If–Else If

You can test multiple conditions within a single **If** block.
This is made possible by adding **Else If** sections:



This block first checks whether **x** is greater than 100, and if so, prints “It is a large number.”
Otherwise, it checks whether **x** is equal to 42, and if so, prints “It is a lucky number.”
If neither condition is met, nothing happens.

An **If** block can have multiple **Else If** sections.
Conditions are evaluated from top to bottom, and execution stops when one condition is satisfied or no conditions remain.

Javascript Code

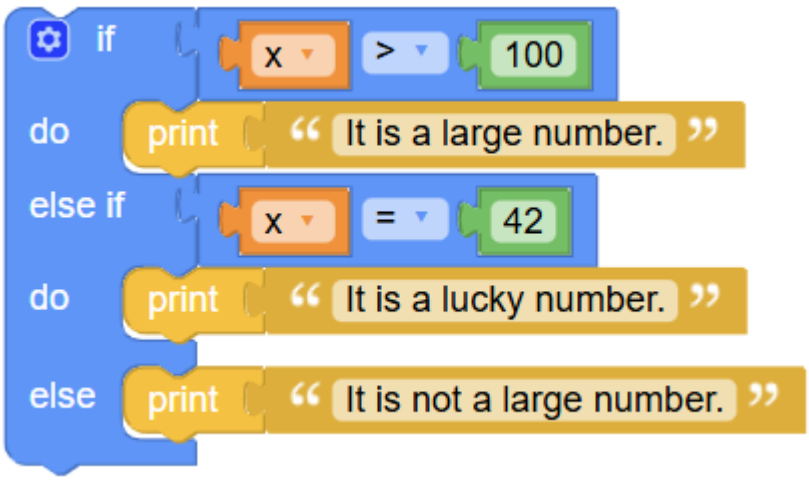
```javascript
if (x > 100) {
    window.alert('It is a large number.');
} else if (x === 42) {
    window.alert('It is a lucky number.');
}
```

Python Code

```python
if x > 100:
    print('It is a large number.')
elif x == 42:
    print('It is a lucky number.')
```

If–Else If–Else

As shown below, an **If** block can have both **Else If** and **Else** sections:



The **Else** section ensures that some action is performed even if all preceding conditions are not true.

The **Else** section can come after any number of **Else If** sections, including zero.

Javascript Code

```javascript
if (x > 100) {
    window.alert('It is a large number.');
} else if (x === 42) {
    window.alert('It is a lucky number.');
} else {
    window.alert('It is not a large number.');
}
```

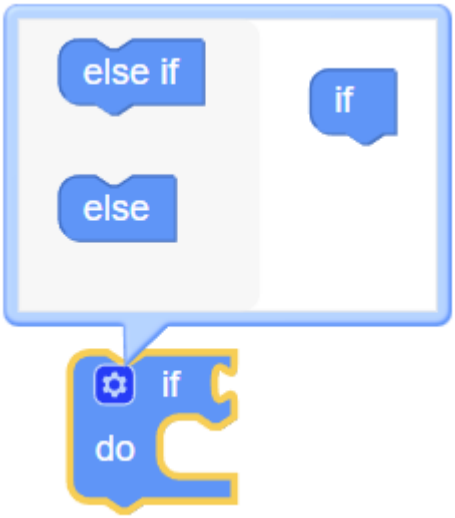
Python Code

```
if x > 100:
    print('It is a large number.')
elif x == 42:
    print('It is a lucky number.')
else:
    print('It is not a large number.')
```

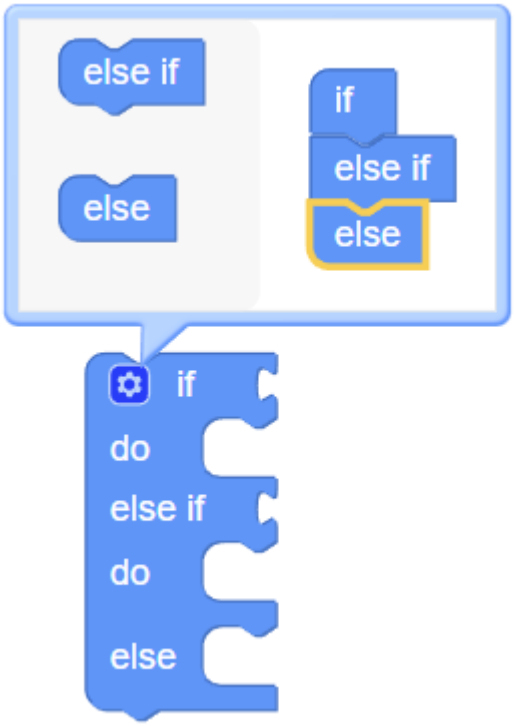
How to Modify Blocks

Only a simple **If** block appears in the toolbox:

To add **Else If** and **Else** clauses, click the gear icon to open a new window:



Drag the **Else If** and **Else** clauses under the **If** block, where you can reorder or remove them. When finished, click the gear icon again to close the window:



You can add as many **Else If** sub-blocks as you want, but note that only **one Else** block can be added.

Loops

There are two types of control blocks:

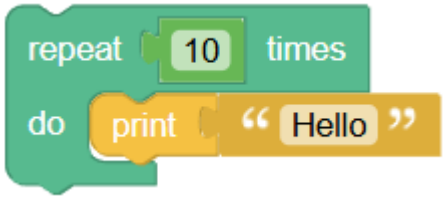
Conditions and **Loops** (which control how many times a block's body is executed, based on variable values)

Loop Blocks

Repeat

The simplest **Repeat** block executes the code inside it a specified number of times.

For example, the block below prints "`Hello!`" ten times.



JavaScript Code

```
for (var count = 0; count < 10; count++) { // repeat 10 times
    window.alert('Hello');
}
```

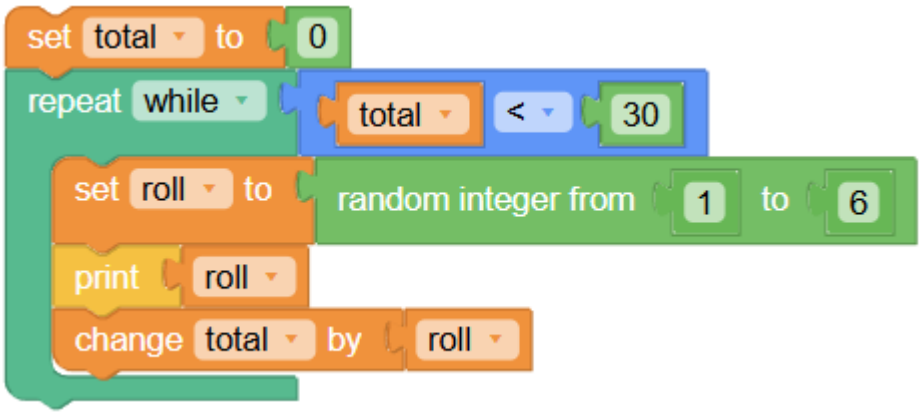
Python Code

```
for count in range(10): # repeat 10 times
    print('Hello')
```

Repeat While

Consider a game where a player rolls a die and keeps adding the values until the total is less than 30. The blocks below implement that game:

1. A variable called **total** is initialized to 0.
2. The loop starts by checking whether **total** is less than 30. If it is, the blocks inside the loop are executed.
3. A random number from 1 to 6 is generated (rolling the die) and stored in a variable called **roll**.
4. The rolled number is displayed.
5. The **total** variable is increased by the value of **roll**.
6. When the loop finishes one iteration, control returns to step 2.



When the loop finishes, the blocks that follow it are executed (omitted here).

In this example, random numbers from 1 to 6 are printed several times, and then the loop ends.

The sum of those numbers is stored in the **total** variable.

This sum will be at least 30.

JavaScript Code

```javascript
var roll, total;

function mathRandomInt(a, b) { // random integer function
    if (a > b) {
        var c = a;
        a = b;
        b = c;
    }
    return Math.floor(Math.random() * (b - a + 1) + a);
}

total = 0;
while (total < 30) { // repeat while total is less than 30
    roll = mathRandomInt(1, 6);
    window.alert(roll);
    total = (typeof total === 'number' ? total : 0) + roll;
    await __wait(10);
}
```

Python Code

```python
import asyncio
import random
from numbers import Number

roll = None
total = None

total = 0
while total < 30:  # repeat while total is less than 30
    roll = random.randint(1, 6)
    print(roll)
```

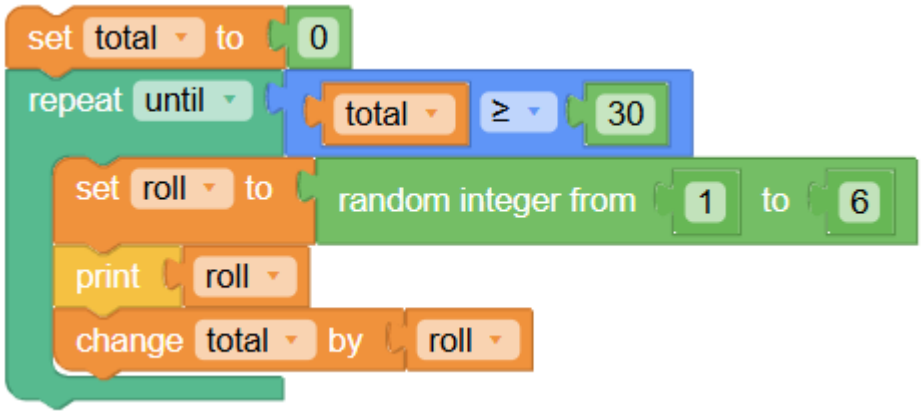
```
    total = (total if isinstance(total, Number) else 0) + roll
    await asyncio.sleep(0.01)
```

Repeat Until

The **Repeat Until** loop executes the body while the condition is false, and exits the loop as soon as the condition becomes true.

The blocks below produce the same result as the previous example.

The loop continues repeating until **total** becomes 30 or greater.



JavaScript Code

```
var total, roll;

function mathRandomInt(a, b) {
    if (a > b) { // random number helper function
        var c = a;
        a = b;
        b = c;
    }
    return Math.floor(Math.random() * (b - a + 1) + a);
}

total = 0;
while (!(total >= 30)) { // repeat until total becomes 30 or greater
    roll = mathRandomInt(1, 6);
```

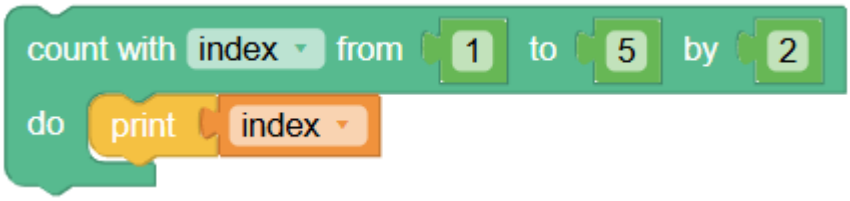
```
    window.alert(roll);
    total = (typeof total === 'number' ? total : 0) + roll;
    await __wait(10);
}
```

Python Code

```
import asyncio
import random
from numbers import Number

total = None
roll = None

total = 0
while not (total >= 30):  # repeat until total becomes 30 or greater
    roll = random.randint(1, 6)
    print(roll)
    total = (total if isinstance(total, Number) else 0) + roll
    await asyncio.sleep(0.01)
```

Count with

The **Count with** block (commonly called a **for loop**) runs the code inside its body once for each value of a variable,
starting from the first value and increasing by the specified step until it reaches the final value. For example, the program below outputs 1, 3, 5.



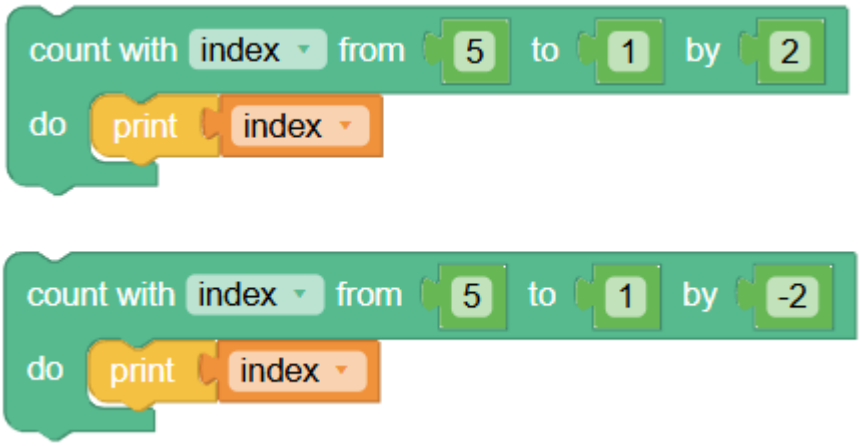
JavaScript Code

```
var index;

for (index = 1; index <= 5; index += 2) {
    window.alert(index);
}
```

Python Code

```
index = None

for index in range(1, 6, 2):
    print(index)
```

The following two loop blocks work even when the first value is greater than the second value. They behave the same way regardless of whether the third value (the step/increment) is positive or negative.



JavaScript Code

```
var index;

for (index = 5; index >= 1; index -= 2) {
    window.alert(index);
}
```

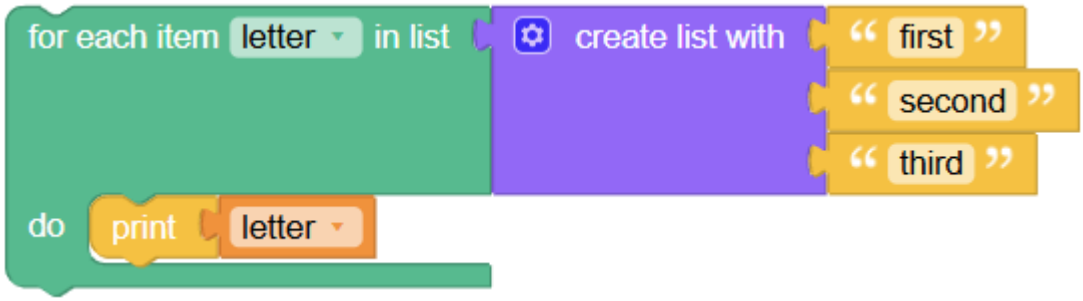
Python Code

```python
index = None

for index in range(5, 0, -2):
    print(index)
```

For Each Item

The **for each item** block works similarly to a loop, but instead of using a numeric counter, it iterates through the values of a list in order.

The program below outputs each item in the list: “first”, “second”, “third”.



JavaScript Code

```javascript
var letter;

var letter_list = ['First', 'Second', 'Third'];
for (var letter_index in letter_list) { // for each item
    letter = letter_list[letter_index];
    window.alert(letter);
}
```

Python Code

```
letter = None

for letter in ['First', 'Second', 'Third']: # for each item
    print(letter)
```

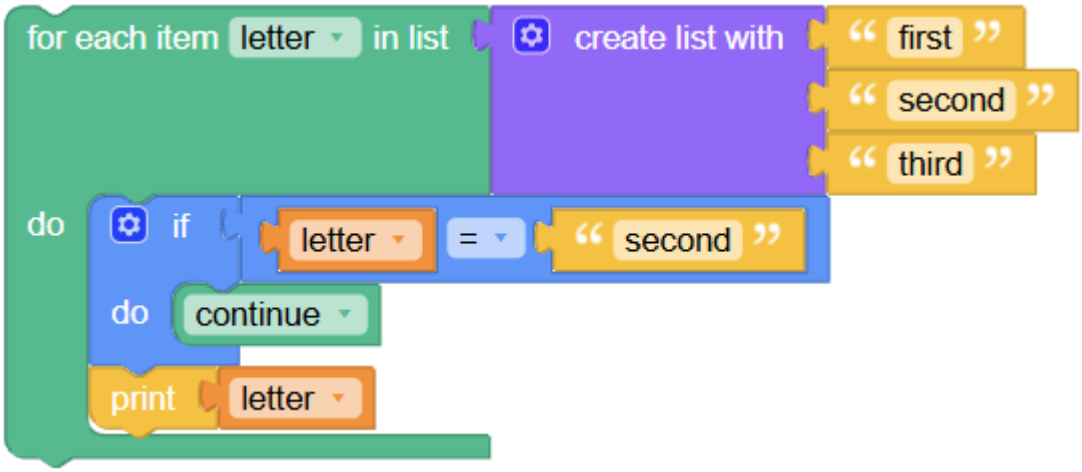
Loop Termination Blocks

Most loops run **until a termination condition is met** (as in conditional loops) or **until all values of the loop variable have been processed** (as in **count with** and **for each item** loops).
Although they are used less frequently, there are two useful blocks that provide additional ways to control loop behavior.
The examples below use a **for each item** loop, but these blocks can be used with all types of loops.

Continue (Next Iteration)

The **continue** block (called **continue** in most programming languages) skips the remaining code in the current loop body and immediately starts the next iteration (pass).

In the following program: During the first iteration, `"First"` is printed. During the second iteration, the **continue** block is executed, so printing `"Second"` is skipped. During the final iteration, `"Third"` is printed.



JavaScript Code

```
var letter;

var letter_list = ['First', 'Second', 'Third'];
for (var letter_index in letter_list) {
    letter = letter_list[letter_index];
    if (letter == 'Second') {
        continue; // skip to the next iteration
    }
    window.alert(letter);
}
```

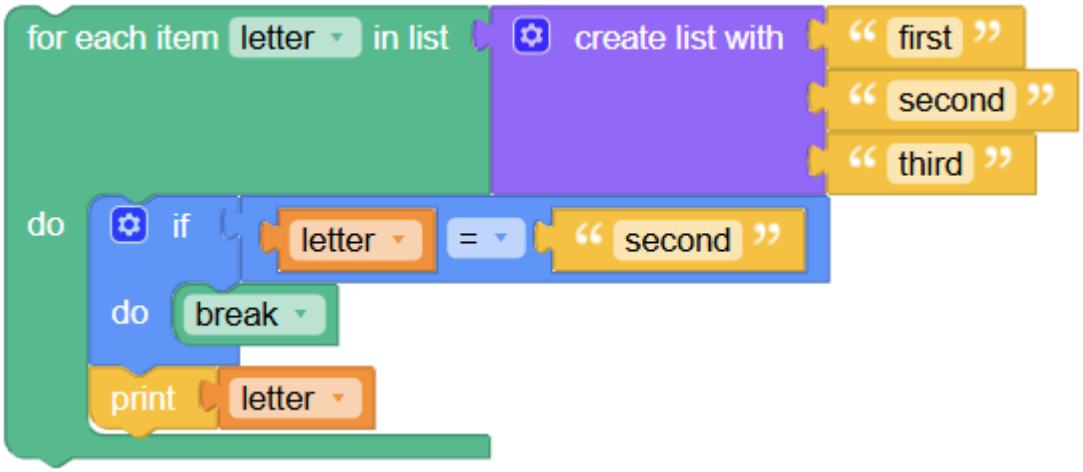
Python Code

```
letter = None

for letter in ['First', 'Second', 'Third']:
    if letter == 'Second':
        continue  # skip to the next iteration
    print(letter)
```

Break

The **Break** block allows you to **exit a loop early**.

In the program below: During the first iteration, `"First"` is printed.
During the second iteration, when the variable value is `"Second"`, the **break loop** block is executed and the loop exits immediately.
As a result, the `"Third"` item in the list is not printed.



JavaScript Code

```
var letter;

var letter_list = ['First', 'Second', 'Third'];
for (var letter_index in letter_list) {
    letter = letter_list[letter_index];
    if (letter == 'Second') {
        break; // break loop
    }
    window.alert(letter);
}
```

Python Code

```
letter = None

for letter in ['First', 'Second', 'Third']:
    if letter == 'Second':
        break  # break loop
    print(letter)
```

Math

This document explains the functions and usage of various calculation blocks.
It introduces blocks that perform a wide range of mathematical operations, including numeric calculations, list processing, probability, and angle operations.

Blocks

Number Value

This block returns the entered **numeric value** as it is.
By using this block, you can store a specific number in a variable or use it in other calculations.

JavaScript Code

```
10; // Numeric value 10
```

Python Code

```
10 # Numeric value 10
```

Creating and Operating on Arrays

This block creates an **array**.
It returns an array whose elements are the values entered inside `[]`.
You can create a list by entering the desired values inside `[]`, and strings must be enclosed in quotation marks `"  "`.

The block above creates `["one", "two", 3]` and outputs one, two, and 3 in order.

JavaScript Code

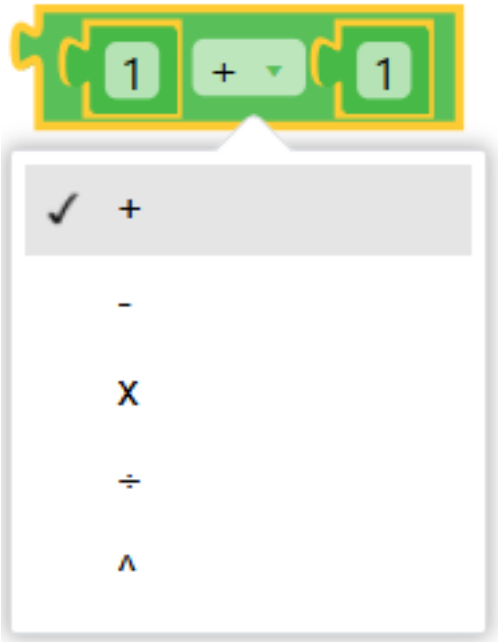
```
var item;

var item_list = ["one", "two", 3 ]; // Create array
for (var item_index in item_list) {
    item = item_list[item_index];
    window.alert(item);
}
```

Python Code

```
item = None

for item in ["one", "two", 3 ]: # Create array
    print(item)
```

Basic Arithmetic Operations

This block performs **arithmetic operations** (addition, subtraction, multiplication, division, and exponentiation) using two numeric values.



The block above calculates 1 + 1 and returns 2.

JavaScript Code

```
1 + 1; // 1 + 1 operation
1 - 1; // 1 - 1 operation
1 * 1; // 1 * 1 operation
1 / 1; // 1 / 1 operation
Math.pow(1, 1); // 1^1 operation
```

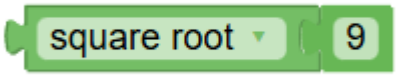
Python Code

```
1 + 1 # 1 + 1 operation
1 - 1 # 1 - 1 operation
1 * 1 # 1 * 1 operation
1 / 1 # 1 / 1 operation
1 ** 1 # 1^1 operation
```

Advanced Operations

Square Root

Returns the **square root** of the given number.



The block above calculates the square root of 9 and returns 3.

JavaScript Code

```
Math.sqrt(9); // Square root of 9
```

Python Code

```
import math

math.sqrt(9) # Square root of 9
```

Absolute Value

Returns the **absolute value** of the given number.

The value of the following block is `10`.

Javascript Code

```
Math.abs(-10); // Absolute value of -10
```

Python Code

```
import math
```

```
math.fabs(-10) # Absolute value of -10
```

Sign

Reverses the **sign** of the input number.

The value of the following block is `-10`.

Javascript Code

```
-10; // Opposite sign of 10
```

Python Code

```
import math

-10 # Opposite sign of 10
```

Exponential and Logarithmic Functions

This block calculates **exponential and logarithmic** values.

It is used to compute powers or logarithms with a specific base.

Javascript Code

```
Math.log(10); // Natural logarithm of 10
Math.log(10)/ Math.log(10); // log base 10 of 10
```

```
Math.exp(2); // e squared
Math.pow(10,2); // 10 squared
```

Python Code

```
import math

math.log(10) # Natural logarithm of 10
math.log10(10) # Logarithm with base 10 and argument 10
math.exp(2) # e squared
math.pow(10,2) # 10 squared
```

Remainder

This block calculates the **remainder** from the division of two numbers.

Javascript Code

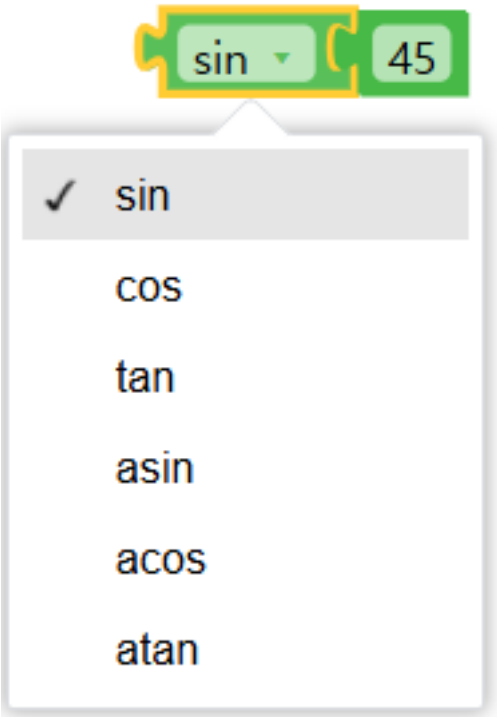
```
64 % 10; // Remainder of 64 ÷ 10
```

Python Code

```
64 % 10 # Remainder of 64 ÷ 10
```

Trigonometric Functions

This block calculates **trigonometric functions** such as sine, cosine, and tangent.



Javascript Code

```
Math.sin(45 / 180 * Math.PI); // sin 45° value
Math.cos(45 / 180 * Math.PI); // cos 45° value
Math.tan(45 / 180 * Math.PI); // tan 45° value
Math.asin(45) / Math.PI * 180; // asin 45° value
Math.acos(45) / Math.PI * 180; // acos 45° value
Math.atan(45) / Math.PI * 180; // atan 45° value
```

Python Code

```
import math
math.sin(45 / 180.0 * math.pi) # sin 45° value
math.cos(45 / 180.0 * math.pi) # cos 45° value
math.tan(45 / 180.0 * math.pi) # tan 45° value
math.asin(45) / math.pi * 180 # asin 45° value
math.acos(45) / math.pi * 180 # acos 45° value
math.atan(45) / math.pi * 180 # atan 45° value
```

round, round down, round up

This block returns a value by performing round, round down, or round up on the input number.

Javascript Code

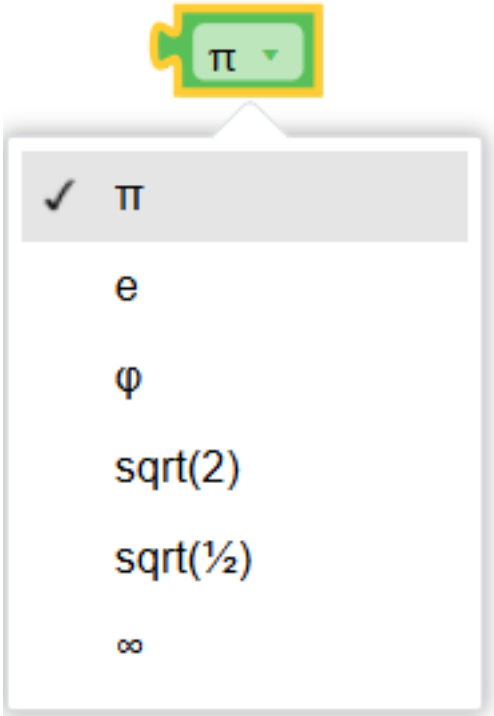
```
Math.round(3.1); // Rounded value of 3.1 is 3
Math.ceil(3.1); // Rounded up value of 3.1 is 4
Math.floor(3.1); // Rounded down value of 3.1 is 3
```

Python Code

```
import math

round(3.1) # Rounded value of 3.1 is 3
math.ceil(3.1) # Rounded up value of 3.1 is 4
math.floor(3.1) # Rounded down value of 3.1 is 3
```

Special Numeric Values

This block provides special **mathematical constants** (such as π and e) and symbols required for calculations.



Javascript Code

```
Math.PI; // π (pi) value
Math.E; // e (Euler's number) value
(1 + Math.sqrt(5)) / 2; // Golden ratio value
Math.SQRT2; // Square root of 2
Math.SQRT1_2; // Square root of 1/2
Infinity; // Infinity
```

Python Code

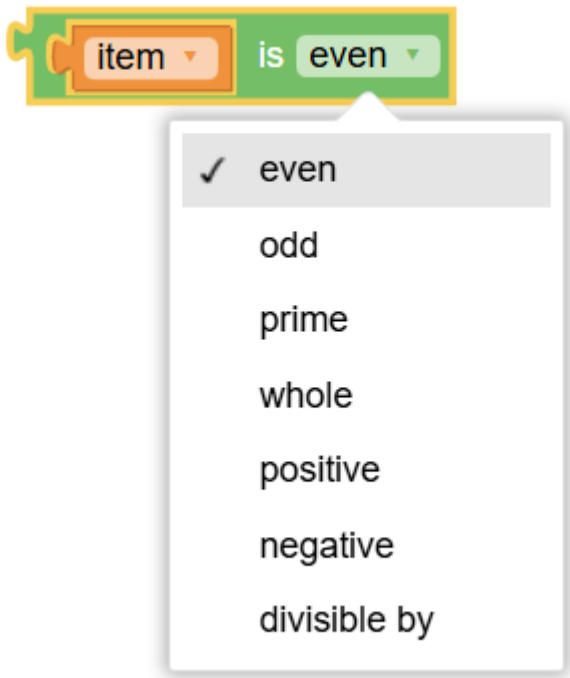
```
import math

math.pi # π (pi) value
math.e # e (Euler's number) value
(1 + math.sqrt(5)) / 2 # Golden ratio value
math.sqrt(2) # Square root of 2
math.sqrt(1.0 / 2) # Square root of 1/2
float('inf') # Infinity
```

Number Conditions

This block checks whether the input number satisfies specific conditions.

It can determine whether a number is even, odd, prime, and more.

If the condition is satisfied, it returns **true**; otherwise, it returns **false**.



Javascript Code

```
var item;

function mathIsPrime(n) { // Prime number check function
    if (n == 2 || n == 3) {
        return true;
    }
    if (isNaN(n) || n <= 1 || n % 1 !== 0 || n % 2 === 0 || n % 3 === 0) {
        return false;
    }
    for (var x = 6; x <= Math.sqrt(n) + 1; x += 6) {
        if (n % (x - 1) === 0 || n % (x + 1) === 0) {
            return false;
```

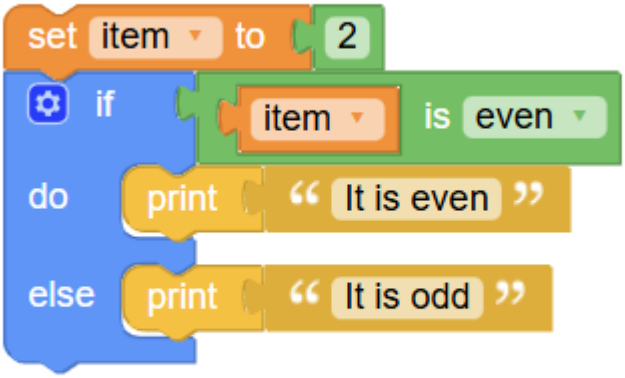
```
        }
    }
    return true;
}

item % 2 === 0; // Condition check: item is even
item % 2 === 1; // Condition check: item is odd
mathIsPrime(item); // Condition check: item is prime
item % 1 === 0; // Condition check: item is an integer
item > 0; // Condition check: item is positive
item < 0; // Condition check: item is negative
item % 0 === 0 // Condition check: item is divisible by 0
```

Python Code

```
import math
from numbers import Number

item = None

def math_isPrime(n): # Prime number check function
    if not isinstance(n, Number):
        try:
            n = float(n)
        except:
            return False
    if n == 2 or n == 3:
        return True
    if n <= 1 or n % 1 != 0 or n % 2 == 0 or n % 3 == 0:
        return False
    for x in range(6, int(math.sqrt(n)) + 2, 6):
        if n % (x - 1) == 0 or n % (x + 1) == 0:
            return False
    return True
```

```
item % 2 == 0 # Condition check: item is even
item % 2 == 1 # Condition check: item is odd
math_isPrime(item) # Condition check: item is prime
item % 1 == 0 # Condition check: item is an integer
item > 0 # Condition check: item is positive
item < 0 # Condition check: item is negative
item % 0 == 0  # Condition check: item is divisible by 0
```

The example below prints the message "`It is even.`" because `item` is even.



Javascript Code

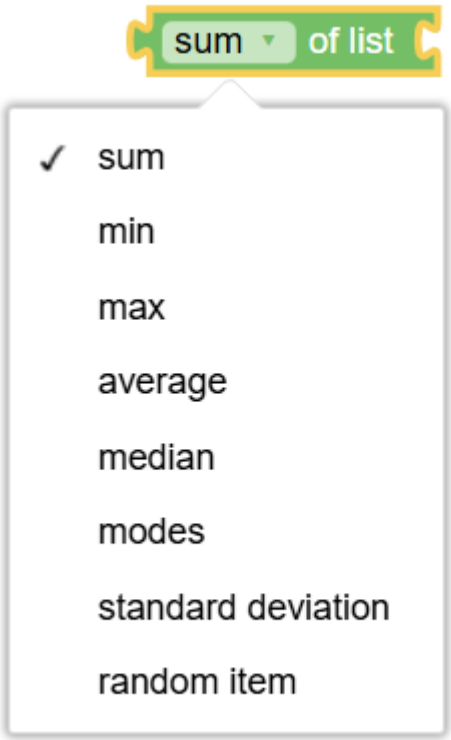
```
var item;

item = 2;
if (item % 2 === 0) {
    window.alert('It is even.');
} else {
    window.alert('It is odd.');
}
```

Python Code

```
item = None

item = 2
if item % 2 == 0:
    print('It is even.')
else:
    print('It is odd.')
```

List Operations

This block performs various operations on a **list**.
It can calculate the sum, minimum value, maximum value, average, median, mode, standard deviation, and select a random item.



Javascript Code

```
function mathMean(myList) { // Mean calculation function
    return myList.reduce(function(x, y) {return x + y;}, 0) / myList.length;
}

function mathMedian(myList) { // Median calculation function
    var localList = myList.filter(function (x) {return typeof x === 'number';});
    if (!localList.length) return null;
    localList.sort(function(a, b) {return b - a;});
    if (localList.length % 2 === 0) {
        return (localList[localList.length / 2 - 1] + localList[localList.length
        / 2]) / 2;
    } else {
        return localList[(localList.length - 1) / 2];
    }
}

function mathModes(values) { // Mode calculation function
    var modes = [];
    var counts = [];
    var maxCount = 0;
    for (var i = 0; i < values.length; i++) {
        var value = values[i];
        var found = false;
        var thisCount;
        for (var j = 0; j < counts.length; j++) {
            if (counts[j][0] === value) {
                thisCount = ++counts[j][1];
                found = true;
                break;
            }
        }
        if (!found) {
            counts.push([value, 1]);
```

```
            thisCount = 1;
        }
        maxCount = Math.max(thisCount, maxCount);
    }
    for (var j = 0; j < counts.length; j++) {
        if (counts[j][1] === maxCount) {
            modes.push(counts[j][0]);
        }
    }
    return modes;
}

function mathStandardDeviation(numbers) { // Standard deviation calculation function
    var n = numbers.length;
    if (!n) return null;
    var mean = numbers.reduce(function(x, y) {return x + y;}) / n;
    var variance = 0;
    for (var j = 0; j < n; j++) {
        variance += Math.pow(numbers[j] - mean, 2);
    }
    variance /= n;
    return Math.sqrt(variance);
}

function mathRandomList(list) { // Random item selection function
    var x = Math.floor(Math.random() * list.length);
    return list[x];
}


[].reduce(function(x, y) {return x + y;}, 0); // List sum
Math.min.apply(null, []); // List minimum value
Math.max.apply(null, []); // List maximum value
```

```
mathMean([]); // List average value
mathMedian([]); // List median value
mathModes([]); // List mode value
mathStandardDeviation([]); // List standard deviation value
mathRandomList([]); // Random item from list
```

Python Code

```
from numbers import Number
import math
import random

item = None

def math_mean(myList): # Mean calculation function
    localList = [e for e in myList if isinstance(e, Number)]
    if not localList: return
    return float(sum(localList)) / len(localList)

def math_median(myList): # Median calculation function
    localList = sorted([e for e in myList if isinstance(e, Number)])
    if not localList: return
    if len(localList) % 2 == 0:
        return (localList[len(localList) // 2 - 1] + localList[len(localList) //
        2]) / 2.0
    else:
        return localList[(len(localList) - 1) // 2]

def math_modes(some_list): # Mode calculation function
    modes = []
    counts = []
    maxCount = 1
    for item in some_list:
        found = False
```

```
        for count in counts:
            if count[0] == item:
                count[1] += 1
                maxCount = max(maxCount, count[1])
                found = True
        if not found:
            counts.append([item, 1])
    for counted_item, item_count in counts:
        if item_count == maxCount:
            modes.append(counted_item)
    return modes

def math_standard_deviation(numbers): # Standard deviation calculation function
    n = len(numbers)
    if n == 0: return
    mean = float(sum(numbers)) / n
    variance = sum((x - mean) ** 2 for x in numbers) / n
    return math.sqrt(variance)

sum([]) # List sum
min([]) # List minimum value
max([]) # List maximum value
math_mean([]) # List average value
math_median([]) # List median value
math_modes([]) # List mode value
math_standard_deviation([]) # List standard deviation value
random.choice([]) # Random item from list
```

The block below calculates the sum of the list [10, 20, 30], so 10 + 20 + 30 = 60.

Javascript Code

```
[10, 20, 30].reduce(function(x, y) {return x + y;}, 0); // Result of the sum of [10, 20, 30] is 60
```

Python Code

```
sum([10, 20, 30]) # Result of the sum of [10, 20, 30] is 60
```

Value Range Limiting

This block restricts the input value so that it does not exceed a specific range. - If the value is smaller than the minimum, it is adjusted to the minimum. - If the value is larger than the maximum, it is adjusted to the maximum.

If the value of `item` is less than 1, it returns 1; if it is greater than 100, it returns 100.

Values between 1 and 100 return the same value.

Javascript Code

```
var item;

Math.min(Math.max(item, 1), 100); // Clamp item to the range 1 ~ 100
```

Python Code

```
item = None

min(max(item, 1), 100) # Clamp item to the range 1 ~ 100
```

Random Integer

This block generates a **random integer** within a specified range.

Generates an integer between 1 and 100 (inclusive).

Javascript Code

```
function mathRandomInt(a, b) { // Random integer generation function
    if (a > b) {
        var c = a;
        a = b;
        b = c;
```

```
    }
    return Math.floor(Math.random() * (b - a + 1) + a);
}

mathRandomInt(1, 100); // Generate a random integer between 1 and 100
```

Python Code

```
import random

random.randint(1, 100) # Generate a random integer between 1 and 100
```

Random Fraction

This block generates a **random fractional value** between 0 and 1.

Javascript Code

```
Math.random(); // Random fractional value
```

Python Code

```
import random

random.random() # Random fractional value
```

Angle

This block calculates the angle formed by the given (x, y) coordinate with the origin (0, 0).
It can be used to determine direction based on the position.

Javascript Code

```
Math.atan2(1, 1) / Math.PI * 180; // Calculated angle value
```

Python Code

```
import math

math.atan2(1, 1) / math.pi * 180 # Calculated angle value
```

Text

Examples of texts include: - “thing #1” - “March 12, 2010” - “” (empty text)

Texts can contain uppercase or lowercase letters, numbers, punctuation, other symbols, and spaces between words.

Blocks

Create Text

The following block creates the text “hello” and stores it in a variable named `greeting`.

Javascript Code

```
var greeting;

greeting = 'hello';
```

Python Code

```
greeting = None

greeting = 'hello'
```

The **Join** block concatenates the value of `greeting` with the text "`world`", producing "`helloworld`". Since neither of the original texts contains a space, the resulting text has no spaces.

Javascript Code

```
var greeting;

greeting = 'hello';
String(greeting) + 'world'; // "helloworld"
```

Python Code

```
greeting = None
```

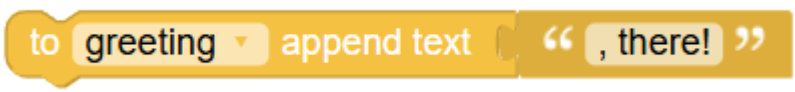
```
greeting = 'hello'
str(greeting) + 'world'  # "helloworld"
```

To increase the number of input texts, click the gear icon. The view will change as shown below:

To add additional inputs, drag the **Item** block from the gray toolbox on the left and insert it into the **Join** block.

Modify Text

The **append text** block adds the given text to the specified variable.
In the example below, the value of the `greeting` variable changes from `"hello"` to `"hello, there!"`.



Javascript Code

```
var greeting;

greeting = 'hello';
greeting += ', there!'; // greeting becomes "hello, there!"
```

Python Code

```
greeting = None

greeting = 'hello'
greeting = str(greeting) + ', there!'  # greeting becomes "hello, there!"
```

Text Length

The **Length** block counts the number of characters (letters, numbers, symbols, etc.) in a text and returns the total length.

For example, the length of "`We're #1!`" is 9, and the length of an empty text is 0.

Javascript Code

```
'We\'re #1!'.length; // Returns the text length of 'We're #1!' → 9
''.length;           // Returns the text length of an empty text → 0
```

Python Code

```
len("We're #1!")  # Returns the text length of "We're #1!" → 9
len('')           # Returns the text length of an empty text → 0
```

Is Empty

The **Is Empty** block checks whether the given text is empty (that is, whether its length is 0).

The first example returns **true**, and the second returns **false**.

Javascript Code

```
var greeting;
```

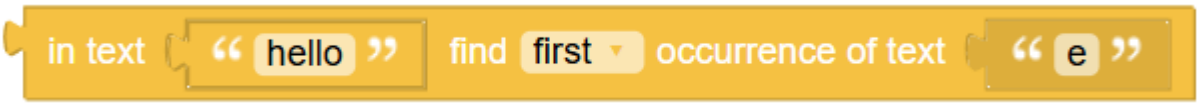
```
greeting = 'hello';
!''.length;        // Returns true because the text is empty
!greeting.length;  // Returns false because the text is not empty
```

Python Code

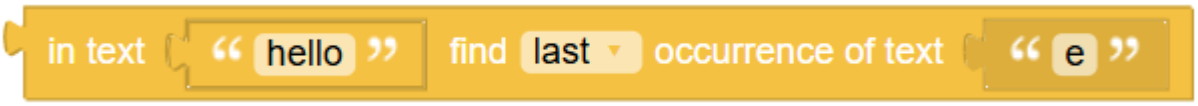
```
greeting = None

greeting = 'hello'
not len('')        # Returns true because the text is empty
not len(greeting)  # Returns false because the text is not empty
```

Finding Text

These blocks check whether a specific text exists within another text and, if it does, return its position. For example, the block below requests the position of the first occurrence of “e” in “hello”, and the result is 2.



The block below requests the position of the last occurrence of “e” in “hello”, and the result is also 2.



Even if the **First** or **Last** option is selected, since “hello” does not contain “z”, the result is 0.

Javascript Code

```
'hello'.indexOf('e') + 1;      // Returns 2: the position where 'e' first appears in the text "hello"
'hello'.lastIndexOf('e') + 1; // Returns 2: the position where 'e' last appears in the text "hello"
'hello'.indexOf('z') + 1;     // Returns 0: 'z' does not exist in the text "hello"
```

Python Code

```
'hello'.find('e') + 1   # Returns 2: the position where 'e' first appears in the text "hello"
'hello'.rfind('e') + 1  # Returns 2: the position where 'e' last appears in the text "hello"
'hello'.find('z') + 1   # Returns 0: 'z' does not exist in the text "hello"
```

Text Extraction

Extracting a Single Character

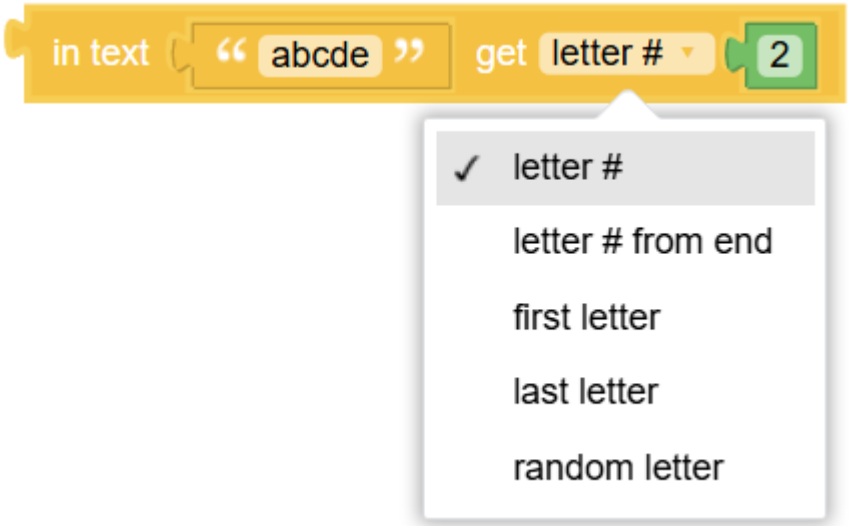
This block retrieves the second character “b” from the text “abcde”.

Javascript Code

```
'abcde'.charAt(1); // Returns the second character 'b' from the text "abcde"
```

Python Code

```
'abcde'[1]  # Returns the second character 'b' from the text "abcde"
```



Javascript Code

```
function textRandomLetter(text) { // Function that returns a random character
   var x = Math.floor(Math.random() * text.length);
   return text[x];
}

'abcde'.charAt(1);            // Get the second character from the front: 'b'
'abcde'.slice(-2).charAt(0);  // Get the second character from the end: 'd'
'abcde'.charAt(0);            // Get the first character: 'a'
'abcde'.slice(-1);            // Get the last character: 'e'
textRandomLetter('abcde');    // Get one random character
```

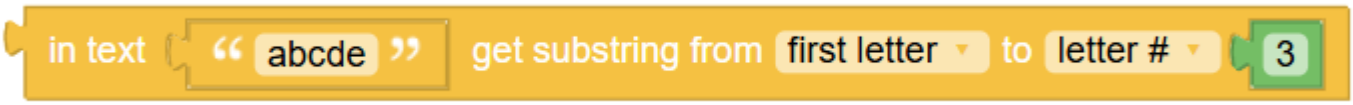
Python Code

```
import random

def text_random_letter(text):  # Function that returns a random character
    x = int(random.random() * len(text))
    return text[x]

'abcde'[1]    # Get the second character from the front: 'b'
'abcde'[-2]   # Get the second character from the end: 'd'
```

```
'abcde'[0]    # Get the first character: 'a'
'abcde'[-1]   # Get the last character: 'e'
text_random_letter('abcde')  # Get one random character
```

Extracting a Subtext

The **Get substring from text** block allows you to extract a specific range of characters from a text.



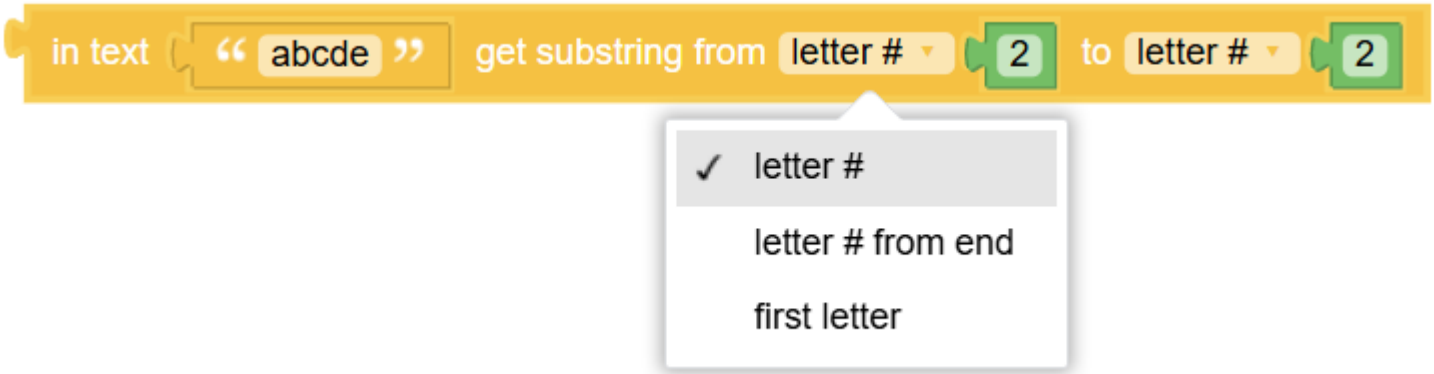
The block above extracts the text **"abc"**.

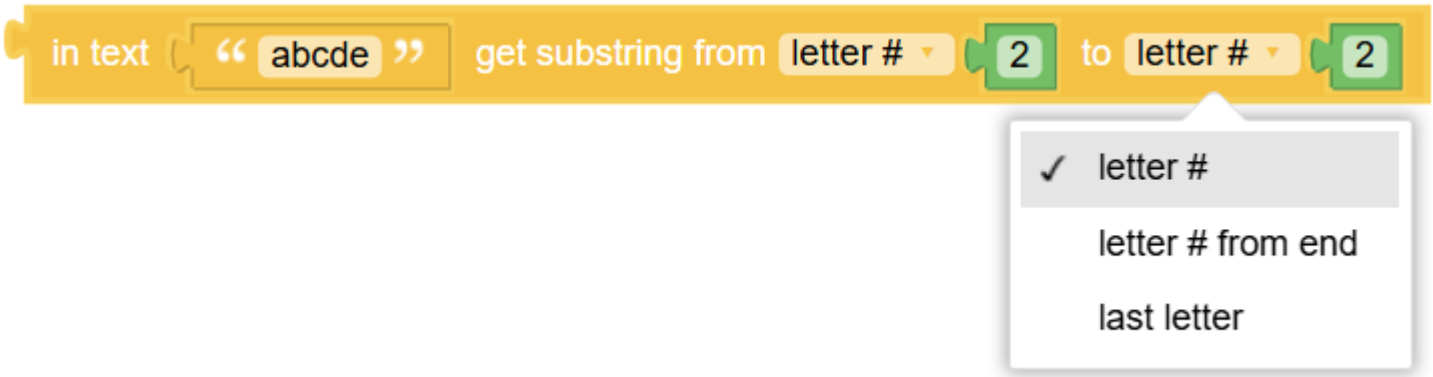
Javascript Code

```
'abcde'.slice(0, 3); // Returns the substring "abc" from "abcde" (from the first to the third character)
```

Python Code

```
'abcde'[:3]  # Returns the substring "abc" from "abcde" (from the first to the third character)
```





Javascript Code

```
'abcde'.slice(1, 4);
// Extracts the substring "bcd" from "abcde" (from the 2nd character to the 4th character)
'abcde'.slice(1, 'abcde'.length - 1);
// Extracts the substring "bcd" from "abcde" (from the 2nd character to the 2nd character from the end)
'abcde'.slice(1, 'abcde'.length);
// Extracts the substring "bcde" from "abcde" (from the 2nd character to the last character)
'abcde'.slice('abcde'.length - 4, 4);
// Extracts the substring "bcd" from "abcde" (from the 4th character from the end to the 4th character)
'abcde'.slice('abcde'.length - 4, 'abcde'.length - 1);
// Extracts the substring "bcd" from "abcde" (from the 4th character from the end to the 2nd character from the end)
'abcde'.slice('abcde'.length - 4, 'abcde'.length);
// Extracts the substring "bcde" from "abcde" (from the 4th character from the end to the last character)
'abcde'.slice(0, 4);
// Extracts the substring "abcd" from "abcde" (from the 1st character to the 4th character)
'abcde'.slice(0, 'abcde'.length - 1);
// Extracts the substring "abcd" from "abcde" (from the 1st character to the 2nd character from the end)
```

```
'abcde';
// Represents the full text "abcde" (from the first to the last character)
```

Python Code

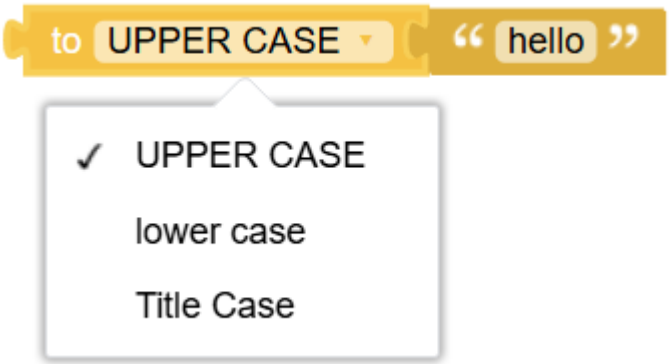
```
'abcde'[1:4]
# Extracts the substring "bcd" from "abcde" (from the 2nd character to the 4th character)
'abcde'[1:-1]
# Extracts the substring "bcd" from "abcde" (from the 2nd character to the 2nd character from the end)
'abcde'[1:]
# Extracts the substring "bcde" from "abcde" (from the 2nd character to the last character)
'abcde'[-4:4]
# Extracts the substring "bcd" from "abcde" (from the 4th character from the end to the 4th character)
'abcde'[-4:-1]
# Extracts the substring "bcd" from "abcde" (from the 4th character from the end to the 2nd character from the end)
'abcde'[-4:]
# Extracts the substring "bcde" from "abcde" (from the 4th character from the end to the last character)
'abcde'[:4]
# Extracts the substring "abcd" from "abcde" (from the 1st character to the 4th character)
'abcde'[:-1]
# Extracts the substring "abcd" from "abcde" (from the 1st character to the 2nd character from the end)
'abcde'[:]
# Extracts the full text "abcde" (from the first to the last character)
```

Change Text Case

This block converts the input text into one of the following formats:

- **Uppercase** (converts all characters to uppercase)
- **Lowercase** (converts all characters to lowercase)
- **Capitalize First Letter** (capitalizes the first letter of each word and lowercases the remaining letters)

The result of the block below is "HELLO".



Javascript Code

```
function textToTitleCase(str) { // Function that converts only the first letter
of each word to uppercase
    return str.replace(/\S+/g,
        function(txt) {
            return txt[0].toUpperCase() + txt.substring(1).toLowerCase();
        });
}

'hello'.toUpperCase();   // Converts all characters in the text to uppercase,
returns 'HELLO'
'hello'.toLowerCase();   // Converts all characters in the text to lowercase,
returns 'hello'
textToTitleCase('hello'); // Converts each word to start with an uppercase letter
and the rest to lowercase, returns 'Hello'
```

Python Code

```
'hello'.upper()  # Converts all characters in the text to uppercase, returns 'HELLO'
'hello'.lower()  # Converts all characters in the text to lowercase, returns 'hello'
'hello'.title()  # Converts each word to start with an uppercase letter and the rest to lowercase, returns 'Hello'
```

Trim Whitespace

This block removes whitespace from specific positions in a text: - The beginning of the text - The end of the text - Both the beginning and the end

The result of the block below is "hi there". (The spaces in the middle of the text are preserved.)

Javascript Code

```
' hi there '.trim();
// Removes whitespace from both sides, returns 'hi there'
' hi there '.replace(/^[\s\xa0]+/, '');
// Removes whitespace from the left side, returns 'hi there '
' hi there '.replace(/[\s\xa0]+$/, '');
// Removes whitespace from the right side, returns ' hi there'
```

Python Code

```python
' hi there '.strip()
# Removes whitespace from both sides, returns 'hi there'
' hi there '.lstrip()
# Removes whitespace from the left side, returns 'hi there '
' hi there '.rstrip()
# Removes whitespace from the right side, returns ' hi there'
```

Count Occurrences of a Substring in a Text

This feature counts how many times a specific word (substring) appears in a given text and returns the count.

Javascript Code

```javascript
function textCount(haystack, needle) { // Function to count occurrences of a specific substring
    if (needle.length === 0) {
        return haystack.length + 1;
    } else {
        return haystack.split(needle).length - 1;
    }
}

textCount('abc, abc, ab', 'abc');
// Returns 2: the number of times "abc" appears in the text "abc, abc, ab"
```

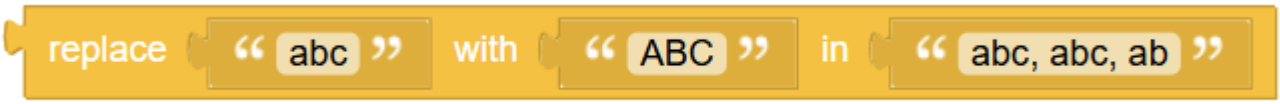
Python Code

```python
'abc, abc, ab'.count('abc')
# Returns 2: the number of times "abc" appears in the text "abc, abc, ab"
```

Returns 2 because the substring "abc" appears twice in the text "abc, abc, ab".

Replace Specific Characters in a Text

This feature replaces a specific character (or substring) in a text with another character of your choice. You can select the target character and substitute it with a new one.



In the text "abc, abc, ab", the substring "abc" is replaced with "ABC".
The result of executing this block is "ABC, ABC, ab".

Javascript Code

```
function textReplace(haystack, needle, replacement) { // Function to replace a specific substring in a text
    needle = needle.replace(/([-()\[\]{}+?*.$\^|,:#<!\\])/g, '\\$1').replace(/\x08/g, '\\x08');
    return haystack.replace(new RegExp(needle, 'g'), replacement);
}

textReplace('abc, abc, ab', 'abc', 'ABC');
// Replaces "abc" with "ABC" in the text "abc, abc, ab", Returns "ABC, ABC, ab"
```

Python Code

```
'abc, abc, ab'.replace('abc', 'ABC')
# Replaces "abc" with "ABC" in the text "abc, abc, ab"
# Returns "ABC, ABC, ab"
```

Reverse Text

This function returns a new text with the characters in reverse order.

The result of reversing "hello" is "olleh".

Javascript Code

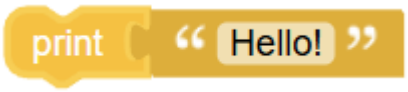
```
'hello'.split('').reverse().join(''); // Returns "olleh", which is the reversed text of "hello"
```

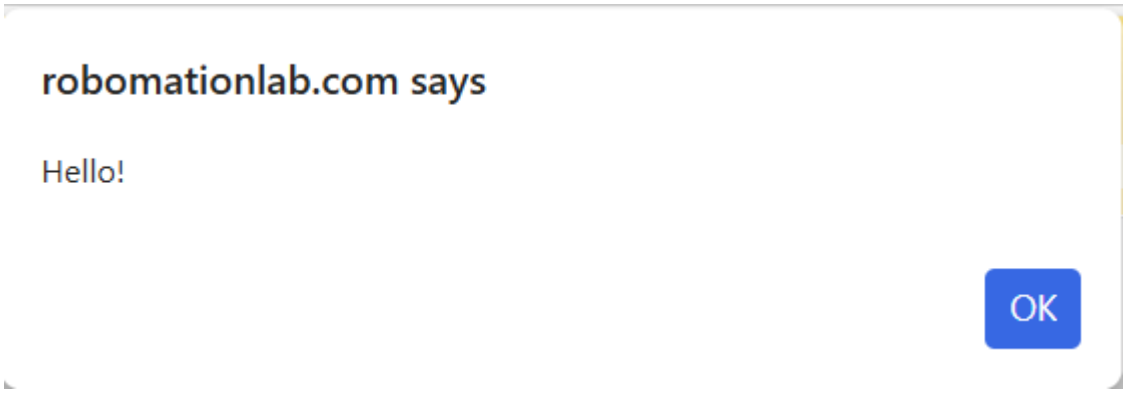
Python Code

```
'hello'[::-1]  # Returns "olleh", which is the reversed text of "hello"
```

Print Text

The **Print** block displays the input value in a pop-up window.





Javascript Code

```
window.alert('Hello!'); // Displays "Hello!" in a pop-up window
```

Python Code

```
print('Hello!') # Displays "Hello!" in a pop-up window
```

Receive User Input

The following block displays a popup window that asks the user to enter their name, and the entered value is stored in the **name** variable.

Javascript Code

```javascript
var name2;

name2 = window.prompt('Please enter your name:');
```

Python Code

```python
name = None

def text_prompt(msg):  # input function
    try:
        return raw_input(msg)
    except NameError:
        return input(msg)

name = text_prompt('Please enter your name:')
```

There is also a version that receives numeric input from the user.

Javascript Code

```javascript
var age;

age = Number(window.prompt('Enter your age:'));
```

Python Code

```python
age = None

def text_prompt(msg):  # function to receive user input
    try:
        return raw_input(msg)
    except NameError:
        return input(msg)

age = float(text_prompt('Enter your age:'))
```

Lists

A **list** is a **collection of items arranged in order**, like a “to-do list” or a “shopping list.”

Items in a list can be of any data type, and the same value can appear multiple times in a list.

Creating Lists

Creating an Empty List

The simplest list is an empty list, which is created using the **create empty list** block:

JavaScript Code

```
[]; // empty list
```

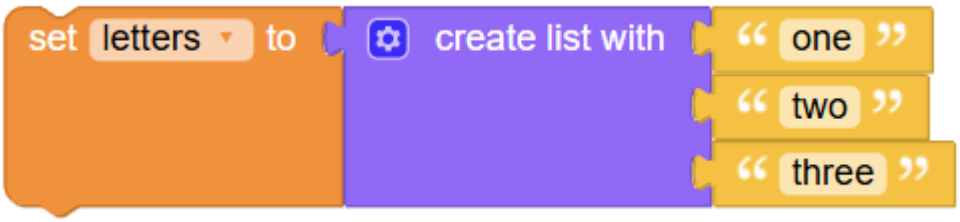
Python Code

```
[]  # empty list
```

Creating a List

Basic Usage

Using the **Create List** block, you can specify initial values for a new list.
For example, you can create a list of words and store it in a variable named **letters**:



In this document, this list is represented as `["one", "two", "three"]`.
The variables defined in this block will be used in later examples.

JavaScript Code

```
var letters;

letters = ['one', 'two', 'three'];
```

Python Code

```
letters = None

letters = ['one', 'two', 'three']
```

The following is an example of creating a list of numeric texts:

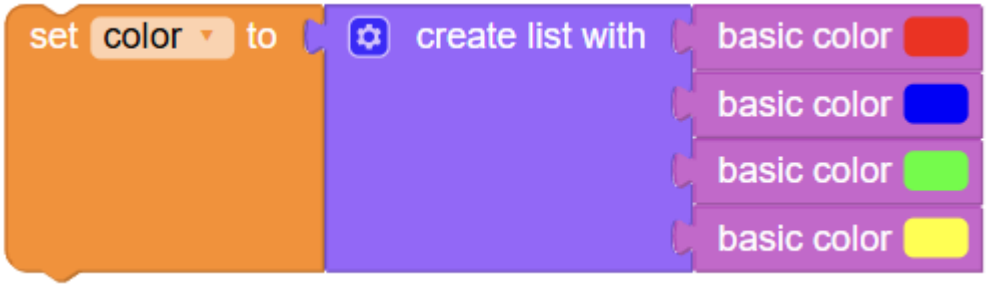
JavaScript Code

```
var numbers;

numbers = ['1', '2', '3'];
```

Python Code

```
numbers = None

numbers = ['1', '2', '3']
```

The following is an example of creating a color list:



JavaScript Code

```
color = [[255, 0, 0], [0, 0, 255], [0, 255, 0], [255, 255, 0]];
```

Python Code

```
color = [[255, 0, 0], [0, 0, 255], [0, 255, 0], [255, 255, 0]]
```

Although less common, you can also create a list that contains values of different types:

JavaScript Code

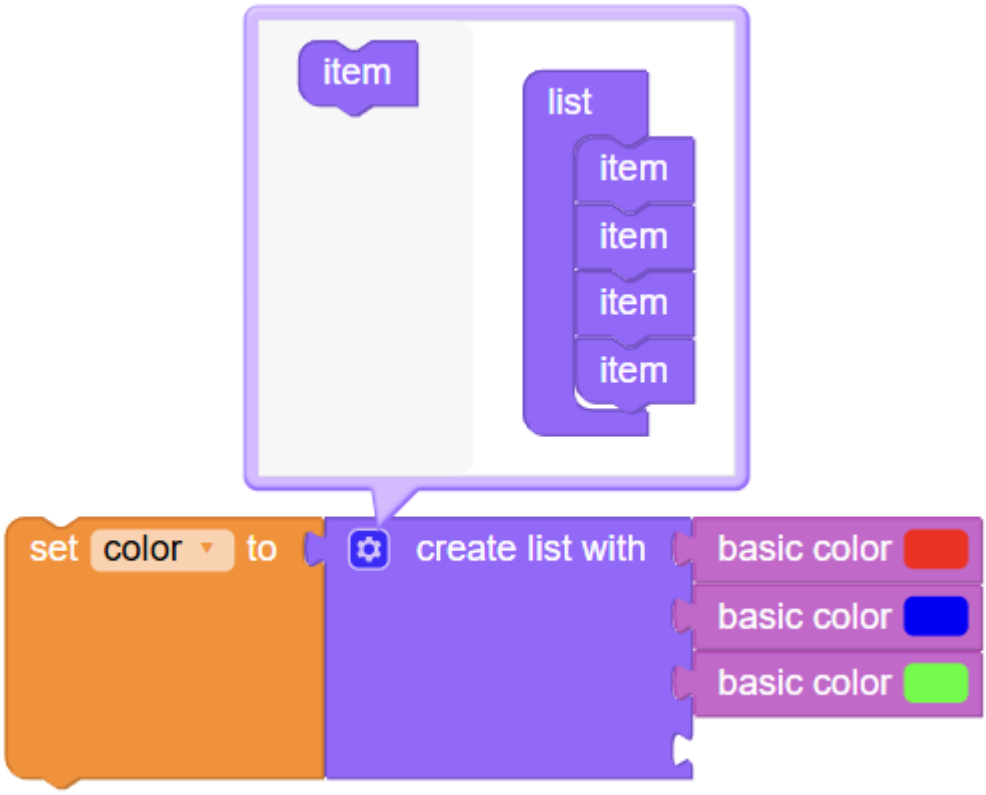
```
['one', 1, [255, 0, 0]];
```

Python Code

```
['one', 1, [255, 0, 0]]
```

Changing the Number of Input Items

To change the number of input items, click the gear icon.
A new window will open, where you can drag the **Item** sub-block on the left into the **List** block on the right to add a new input.



New items can be added at any desired position. Likewise, you can remove unwanted **Item** sub-blocks by dragging them back to the left.

Create List with Repeated Item

The **Create List with Repeated Itemm** block allows you to create a list by repeating a specified item a given number of times.
For example, the block below sets the variable **words** to `["very", "very", "very"]`.

JavaScript Code

```
var words;

function listsRepeat(value, n) { // Function to create a list by repeating the same value
```

```
    var array = [];
    for (var i = 0; i < n; i++) {
        array[i] = value;
    }
    return array;
}

words = listsRepeat('very', 3); // words = ['very', 'very', 'very']
```

Python Code

```
words = None
words = ['very'] * 3 # words = ['very', 'very', 'very']
```

List Length

Is Empty

The value of the **Is Empty** block is **true** if the input is an empty list, and **false** otherwise (this also applies if the input is not a list).

In the example below, the value of the block becomes **false** because the **color** variable is not empty and contains three items.

JavaScript Code

```
var color;

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0]];
!color.length; // Outputs false because it is not empty
```

Python Code

```python
color = None

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0]]
not len(color)  # Outputs False because it is not empty
```

This is similar to the “Is Empty” block in Text category.

Length

The value of the **Length** block is the number of items in the list.

For example, in the block below, **color** contains 3 items, so it returns 3.

JavaScript Code

```javascript
var color;

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0]];
color.length; // Returns 3 because the list contains three items
```

Python Code

```python
color = None

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0]]
len(color)  # Returns 3 because the list contains three items
```

The **Length** block tells you how many items are in a list.

It counts the number of elements in the list itself, not how many different values it contains.

For example, the **words** list below contains the text “very” repeated three times ([“very”, “very”, “very”]).

In this case, the block returns 3.

This is similar to the “Text Length” block in Text category.

JavaScript Code

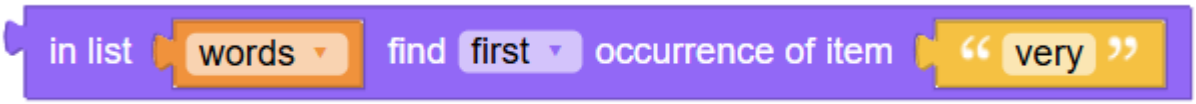
```
var words;

function listsRepeat(value, n) { // Function to create a list by repeating the
same value
    var array = [];
    for (var i = 0; i < n; i++) {
        array[i] = value;
    }
    return array;
}

words = listsRepeat('very', 3);
words.length; // Returns 3 because there are 3 items
```

Python Code

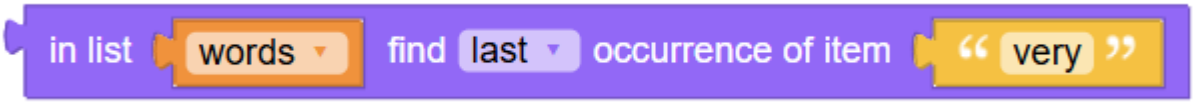
```
words = None

words = ['very'] * 3
len(words)  # Returns 3 because there are 3 items
```

Item Position in a List

These blocks find the position of an item within a list.
For example, the value of the block below is 1 because the first occurrence of the item “very” appears at the first position in the **words** list ([“very”, “very”, “very”]).



The following result is 3 because the item “very” appears at the last position in the **words** list.



If the item does not exist in the list, the result is 0.

These blocks are similar to the “Find Text” block in Text category.

JavaScript Code

```
var words;

function listsRepeat(value, n) { // Function to repeat the same value in a list
    var array = [];
    for (var i = 0; i < n; i++) {
        array[i] = value;
    }
    return array;
}

words = listsRepeat('very', 3);
words.indexOf('very') + 1      // Returns 1: the first position where "very"
appears
words.lastIndexOf('very') + 1 // Returns 3: the last position where "very" appears
words.indexOf('unicorn') + 1  // Returns 0: "unicorn" does not appear in the list
```

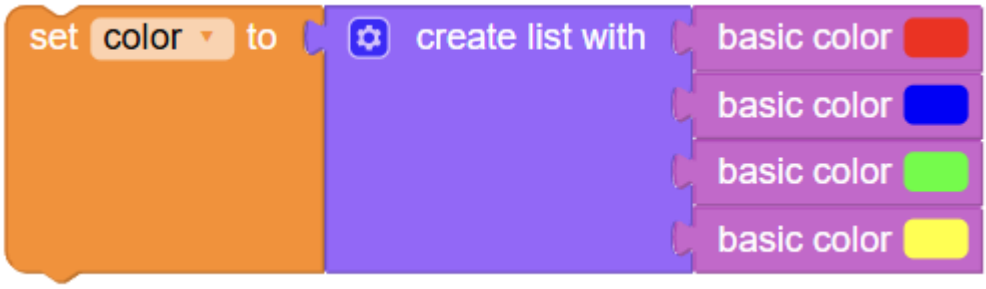
Python Code

```
words = None

def first_index(my_list, elem):  # Function to find the first occurrence
    try:
        index = my_list.index(elem) + 1
    except:
        index = 0
    return index

def last_index(my_list, elem):  # Function to find the last occurrence
    try:
        index = len(my_list) - my_list[::-1].index(elem)
    except:
        index = 0
    return index

words = ['very'] * 3
first_index(words, 'very')    # Returns 1: the first position where "very" appears
last_index(words, 'very')     # Returns 3: the last position where "very" appears
first_index(words, 'unicorn') # Returns 0: "unicorn" does not appear in the list
```

Get Items from a List

Get a Single Item

Remember the definition of the **color** list:



The following block retrieves the color `blue`.

This is because `blue` is the second item in the list (counting from the left):

The next block retrieves `green`.

This is because `green` is the second item from the right end of the list:

This block retrieves the first item, `red`:

This block retrieves the last item, `yellow`:

This block selects one item from the list at random.

It chooses one of `red`, `blue`, `green`, or `yellow` with equal probability.

JavaScript Code

```
var color;

function listsGetRandomItem(list, remove) { // Function to select a random item
from a list
    var x = Math.floor(Math.random() * list.length);
    if (remove) {
        return list.splice(x, 1)[0];
    } else {
        return list[x];
```

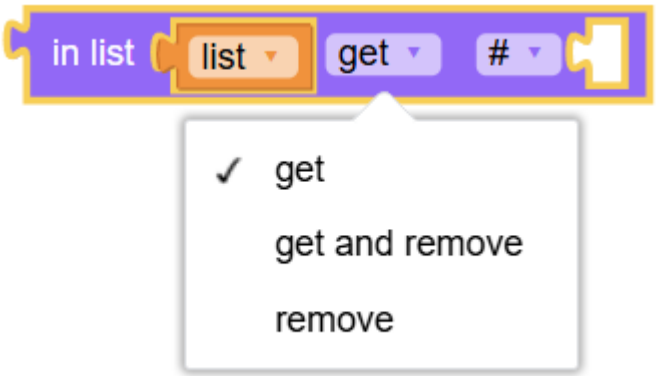
```
    }
}

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0], [255, 255, 0]];
color[1]; // Get the second item (blue) from color
color.slice(-2)[0]; // Get the second item from the end (green) from color
color[0]; // Get the first item (red) from color
color.slice(-1)[0]; // Get the last item (yellow) from color
listsGetRandomItem(color, false); // Get a random item from color
```

Python Code

```
import random

color = None

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0], [255, 255, 0]]
color[1]        # Get the second item (blue) from color
color[-2]       # Get the second item from the end (green) from color
color[0]        # Get the first item (red) from color
color[-1]       # Get the last item (yellow) from color
random.choice(color)  # Get a random item from color
```

Get and Remove Item

When you select an option from the dropdown menu in the **Get item from list** block, it changes to **Get and remove item from list**,
which not only retrieves the item but also modifies the original list by removing that item.



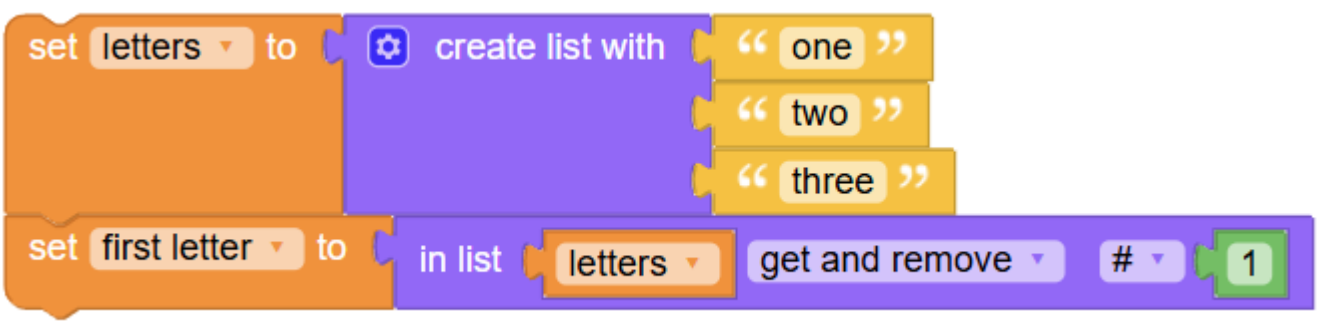
JavaScript Code

```
var list;

function listsGetRandomItem(list, remove) { // Function to get a random item from
a list
   var x = Math.floor(Math.random() * list.length);
   if (remove) {
      return list.splice(x, 1)[0];
   } else {
      return list[x];
   }
}

list[0]; // Get the first item from the list
list.slice(-1)[0]; // Get the last item from the list
listsGetRandomItem(list, false); // Get a random item from the list
list.splice(0, 1)[0]; // Remove and return the first item from the list
list.shift(); // Remove the first item from the list (returns the removed item)
list.pop(); // Remove and return the last item from the list
listsGetRandomItem(list, true); // Remove and return a random item from the list
```

Python Code

```
list2 = None  # Use list2 to avoid shadowing Python's built-in type name list.

def lists_remove_random_item(myList):  # Function to remove a random item from the list
    x = int(random.random() * len(myList))
    return myList.pop(x)

list2[0]        # Get the first item in the list
list2[-1]       # Get the last item in the list
random.choice(list2)  # Get a random item from the list
list2.pop(0)    # Remove the first item from the list
list2.pop(0)    # Remove an item from the front of the list (returns the first item)
list2.pop()     # Remove the last item from the list
lists_remove_random_item(list2)  # Remove a random item from the list
```

This example assigns ["one"] to the variable **first_letter**, and leaves **letters** as ["two", "three"].



JavaScript Code

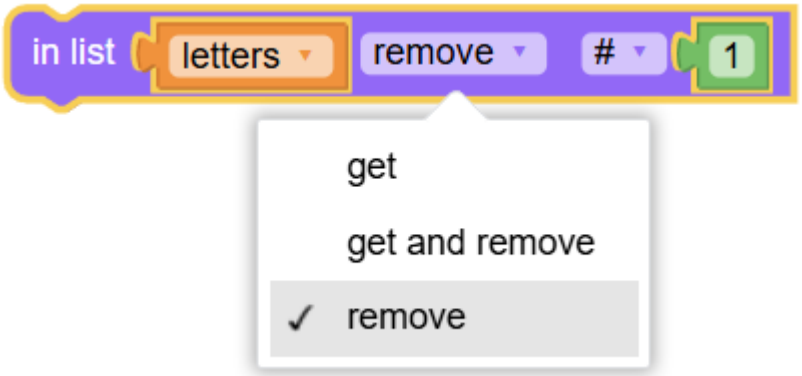
```
var letters, first_letter;

letters = ['one', 'two', 'three'];
first_letter = letters.splice(0, 1)[0];
// Remove the first item from the letters list
// first_letter = ["one"], letters = ["two", "three"]
```

Python Code

```
letters = None
first_letter = None

letters = ['one', 'two', 'three']
first_letter = letters.pop(0)
# Remove the first item from the letters list
# first_letter = ["one"], letters = ["two", "three"]
```

Remove Item

When you select “delete” from the dropdown menu, the plug on the left side of the block disappears:



This removes the first item from **letters**.

JavaScript Code

```
var letters;

letters = ['one', 'two', 'three'];
letters.splice(0, 1); // Remove the first item from the letters list, letters =
['two', 'three']
```

Python Code

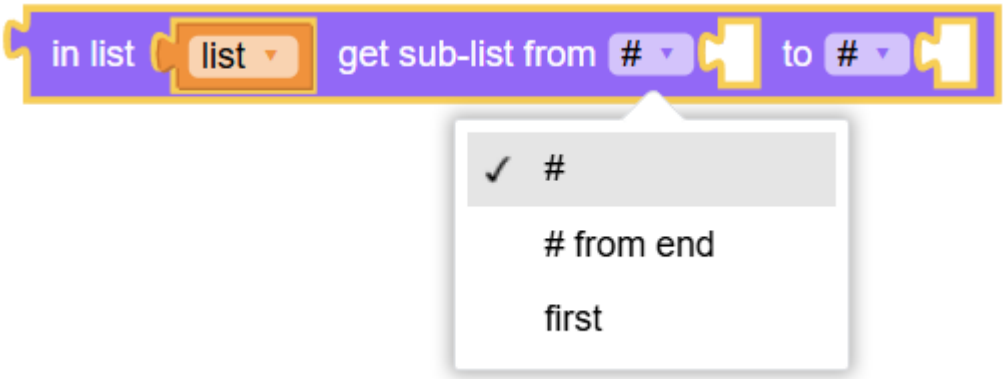
```
letters = None
```

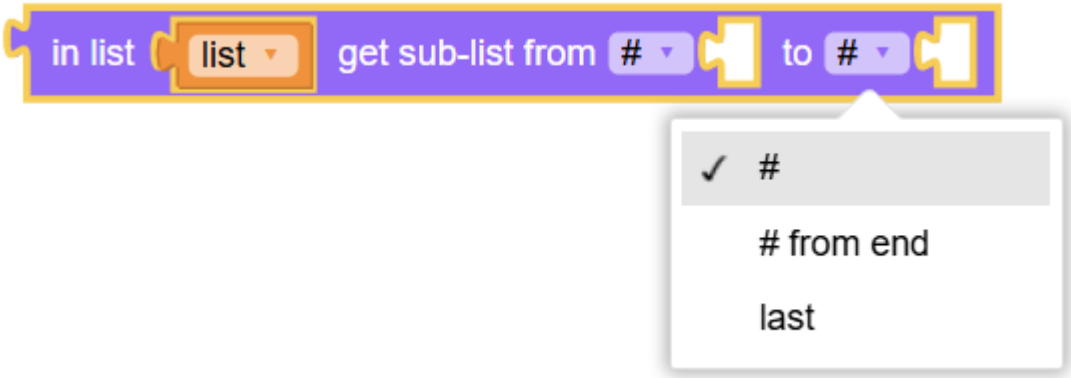
```
letters = ['one', 'two', 'three']
letters.pop(0)  # Remove the first item from the letters list, letters = ['two',
'three']
```

Extracting a Sublist

The **extract sublist from list** block is similar to the **get item from list** block, but instead of extracting a single item, it extracts a sublist.

There are several ways to specify the start and end positions of the sublist:





JavaScript Code

```
var list;

list.slice(0, 1); // Extract a sublist from the first position to the first item
list.slice(0, list.length - 0); // Extract a sublist from the first position to
the last item
list.slice(0, list.length); // Extract a sublist from the first position to the
last item
```

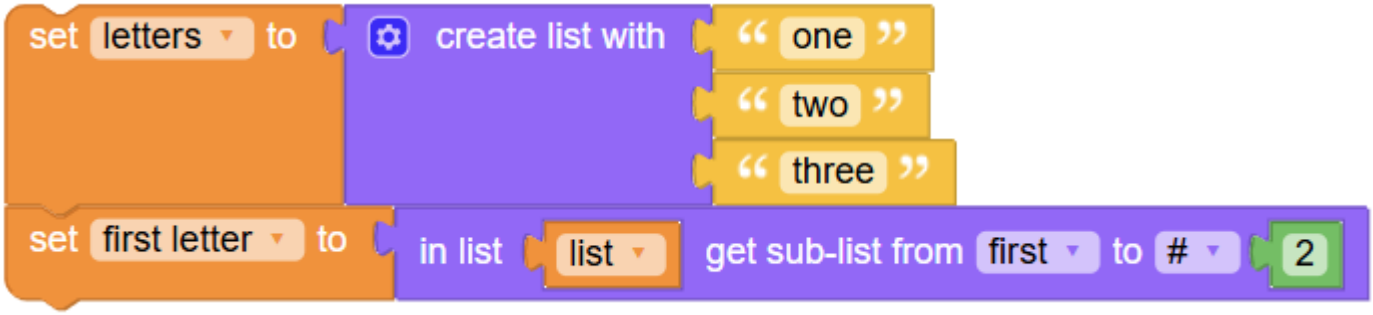
```
list.slice(list.length - 1, 1); // Extract a sublist from the last position to
the last item
list.slice(list.length - 1, list.length - 0); // Extract a sublist from the last
position to the last item
list.slice(list.length - 1, list.length); // Extract a sublist from the last
position to the last item
list.slice(0, 1); // Extract a sublist from the first position to the first item
list.slice(0, list.length - 0); // Extract a sublist from the first position to
the end
list.slice(0); // Extract a sublist from the first position to the last item
```

Python Code

```
list2 = None  # Use list2 because 'list' is a reserved word

list2[:1]    # Extract a sublist from the first position to the first item
list2[:]     # Extract a sublist from the first position to the end
list2[:]     # Extract a sublist from the first position to the last item
list2[-1:1]  # Extract a sublist from the last position to the end
list2[-1:]   # Extract a sublist from the last position to the end
list2[-1:]   # Extract a sublist from the last position to the last item
list2[:1]    # Extract a sublist from the first position to the first item
list2[:]     # Extract a sublist from the first position to the end
list2[:]     # Extract a sublist from the first position to the last item
```

In this example, a new list called **first letters** is created.

This list contains two items: [“one”, “two”].



This block does not modify the original list.

JavaScript Code

```
var letters, first_letter, list;

letters = ['one', 'two', 'three'];
first_letter = list.slice(0, 2); // Extract from the first position to the second
item → first_letter = ["one", "two"]
```

Python Code

```
letters = None
first_letter = None
list2 = None  # list is a reserved word in Python, so list2 is used

letters = ['one', 'two', 'three']
first_letter = letters[:2]  # Extract from the first position to the second item
→ first_letter = ["one", "two"]
```

Adding Items to a List

Set item in list

The **Set item in list** block replaces the item at a specified position in a list with another item.

JavaScript Code

```
var letters, list;

list[0] = 'hello'; // Set "hello" at the first position in the list
list[list.length - 1] = 'hello'; // Set "hello" at the last position in the list
list[0] = 'hello'; // Set "hello" as the first item in the list
list[list.length - 1] = 'hello'; // Set "hello" as the last item in the list
var tmpX = Math.floor(Math.random() * list.length); // Calculate a random index
list[tmpX] = 'hello'; // Set "hello" at a random position in the list
```

Python Code

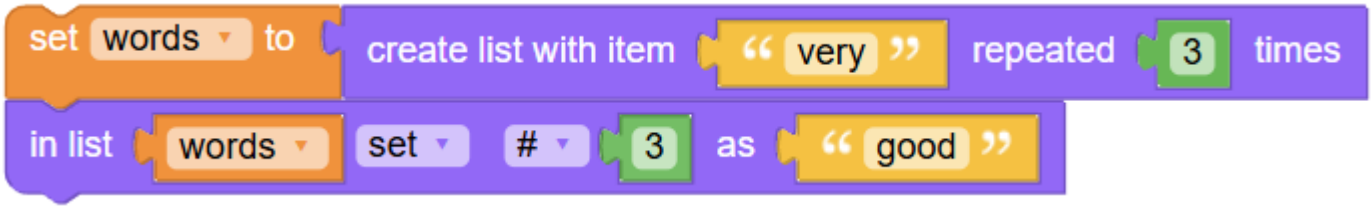
```
import random

letters = None
list2 = None  # Use list2 to avoid shadowing Python’s built-in type name list.

list2[0] = 'hello'        # Set the text "hello" at the first position in the list
list2[-1] = 'hello'       # Set the text "hello" at the last position in the list
list2[0] = 'hello'        # Set "hello" as the first item in the list
list2[-1] = 'hello'       # Set "hello" as the last item in the list
tmp_x = int(random.random() * len(list2))  # Calculate a random index
```

```
list2[tmp_x] = 'hello'    # Set "hello" at a random position in the list
```

The meaning of the dropdown options can be found in the previous section.

The following example does two things: 1. It creates the **words** list as [“very”, “very”, “very”]. 2. It replaces the third item in the list with “good”.

The new value of **words** becomes [“very”, “very”, “good”].



JavaScript Code

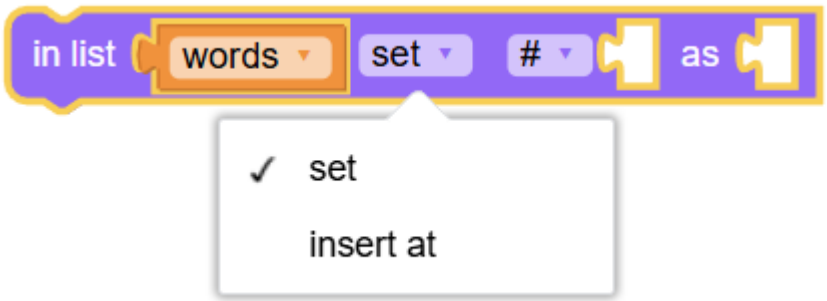
```
var words;

function listsRepeat(value, n) { // Function to create a list by repeating the
same value
    var array = [];
    for (var i = 0; i < n; i++) {
        array[i] = value;
    }
    return array;
}

words = listsRepeat('very', 3); // words = ["very", "very", "very"]
words[2] = 'good';              // words = ["very", "very", "good"]
```

Python Code

```
words = None

words = ['very'] * 3 # words = ['very','very','very']
words[2] = 'good'# words = ['very','very','good']
```

Insert item at a specific position in a list

The **Insert item at a specific position in a list** block can be obtained by selecting the appropriate option from the dropdown menu of the **Set item in list** block.



JavaScript Code

```
var words;

words.splice(2, 0, 'good'); // Insert the text "good" before the third position
words.splice(words.length - 3, 0, 'good'); // Insert the text "good" before the
third position from the end
words.unshift('good'); // Insert the text "good" at the beginning
words.push('good'); // Insert the text "good" at the last position
var tmpX = Math.floor(Math.random() * words.length); // Calculate a random
position
words.splice(tmpX, 0, 'good'); // Insert the text "good" at a random position
```

Python Code

```
import random

words = None

words.insert(2, 'good')  # Insert the text "good" before the third position
words.insert(-3, 'good') # Insert the text "good" before the third position from
the end
words.insert(0, 'good')  # Insert the text "good" at the beginning
```

```
words.append('good')     # Insert the text "good" at the last position
tmp_x = int(random.random() * len(words))  # Calculate a random position
words.insert(tmp_x, 'good')  # Insert the text "good" at a random position
```

This block inserts a new item into a specified position in a list.

For example, in the example below, three things happen:

1. The **words** list is created as [“very”, “very”, “very”].
2. The third item in the list is replaced with “good”. The new value becomes [“very”, “very”, “good”].
3. The word “You’re” is inserted at the beginning of the list. The final value becomes [“You’re”, “very”, “very”, “good”].

JavaScript Code

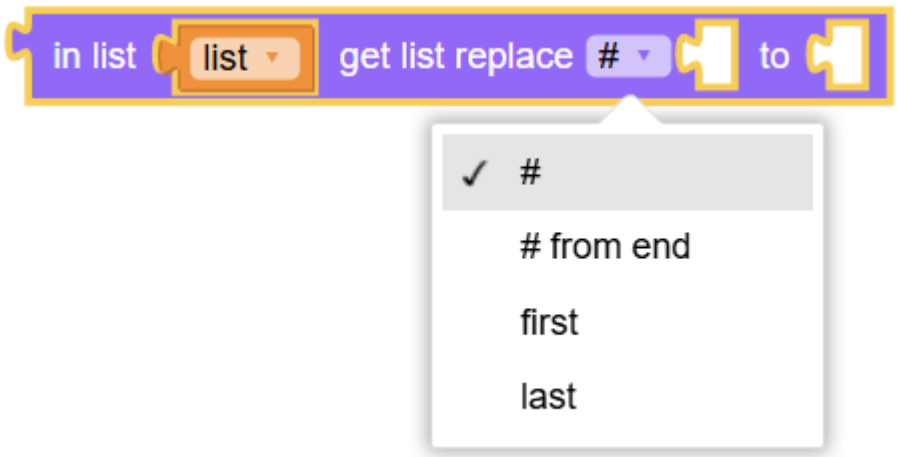
```
var words;

function listsRepeat(value, n) { // Function to create a list with repeated values
   var array = [];
   for (var i = 0; i < n; i++) {
      array[i] = value;
   }
   return array;
}

words = listsRepeat('very', 3);
words[2] = 'good'; // words = ["very", "very", "good"]
words.splice(0, 0, "You're"); // words = ["You're", "very", "very", "good"]
```

Python Code

```
words = None

words = ['very'] * 3
words[2] = 'good'  # words = ['very', 'very', 'good']
words.insert(0, "You're")  # words = ["You're", 'very', 'very', 'good']
```

Replace List Values and Return List

The **"extract a list with item ... replaced by ..."** block replaces the item at a specified position with another item and then returns the entire list.



JavaScript Code

```
var words;

words.map((data, idx) => (idx === 0 ? 'good' : data));
// Extract a new list where the first item of the words list is replaced with
"good"

words.map((data, idx) => (idx === words.length - 1 ? 'good' : data));
// Extract a new list where the last item of the words list is replaced with
"good"
```

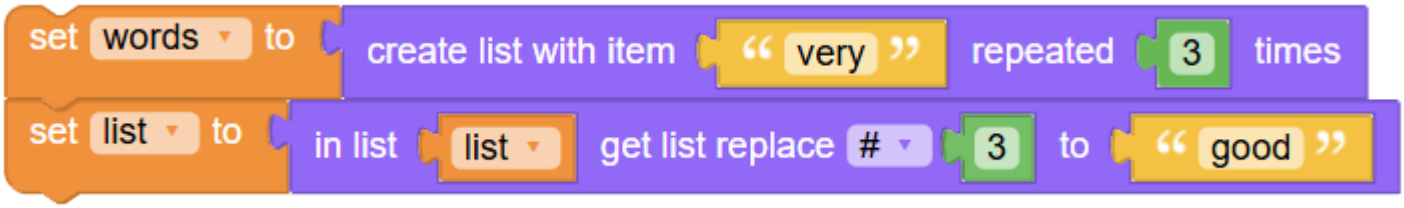
```
words.map((data, idx) => (idx === 0 ? 'good' : data));
// Extract a new list where the first item of the words list is replaced with
"good"

words.map((data, idx) => (idx === words.length - 1 ? 'good' : data));
// Extract a new list where the last item of the words list is replaced with
"good"
```

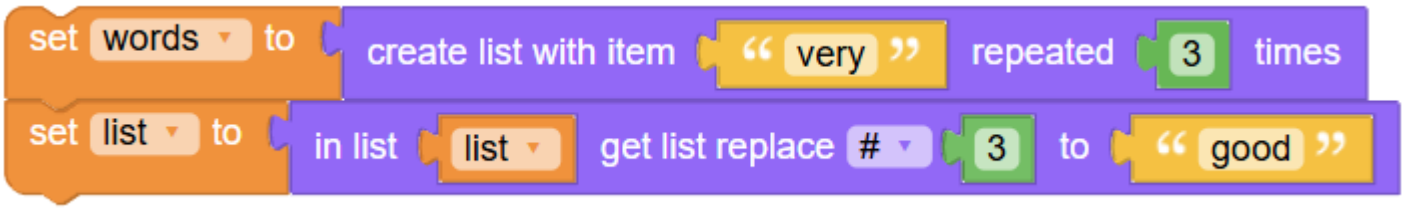
Python Code

```
words = None

['good' if i == 0 else data for i, data in enumerate(words)]
# Extract a new list where the first item of the words list is replaced with
"good"

['good' if i == len(words) - 1 else data for i, data in enumerate(words)]
# Extract a new list where the last item of the words list is replaced with "good"

['good' if i == 0 else data for i, data in enumerate(words)]
# Extract a new list where the first item of the words list is replaced with
"good"

['good' if i == len(words) - 1 else data for i, data in enumerate(words)]
# Extract a new list where the last item of the words list is replaced with "good"
```

Refer to the previous section for the meaning of the dropdown options.

The following example performs three actions: 1. Creates the **words** list as [“very”, “very”, “very”]. 2. Returns a new list in which the third item is replaced with “good”, and assigns this list to a new variable **list**. 3. The **list** list becomes [“very”, “very”, “good”]. The value of the **words** list remains unchanged.





JavaScript Code

```
var words, list;

function listsRepeat(value, n) { // Function to create a list by repeating the
same value
    var array = [];
    for (var i = 0; i < n; i++) {
        array[i] = value;
    }
    return array;
}

words = listsRepeat('very', 3); // words = ["very", "very", "very"]
list = (list.map((data, idx) => (idx === 2 ? 'good' : data))); // list = ["very",
"very", "good"]
```

Python Code

```
words = None
list2 = None  # Use list2 to avoid shadowing Python’s built-in type name list.

words = ['very'] * 3  # words = ['very', 'very', 'very']
list2 = ['good' if i == 2 else data for i, data in enumerate(words)]
```

```
# list2 = ['very', 'very', 'good']
```

Splitting Texts and Combining Lists

Create a List from Text

The **Create List from Text** block splits the given text into pieces using a specified delimiter:

JavaScript Code

```
'311-555-2368'.split('-'); // Split using '-' as the delimiter, returns ["311",
"555", "2368"]
```

Python Code

```
'311-555-2368'.split('-')  # Split using '-' as the delimiter, returns ["311",
"555", "2368"]
```

In the example above, the new list is split into “311”, “555”, and “2368”.

Make Text from List

The **Make Text from List** block joins the items of a list into a single text text using a specified delimiter:

JavaScript Code

```
['311', '555', '2368'].join('-'); // Join the list into a single text using '-' as the delimiter: "311-555-2368"
```

Python Code

```
'-'.join(['311', '555', '2368'])  # Join the list into a single text using '-' as the delimiter: "311-555-2368"
```

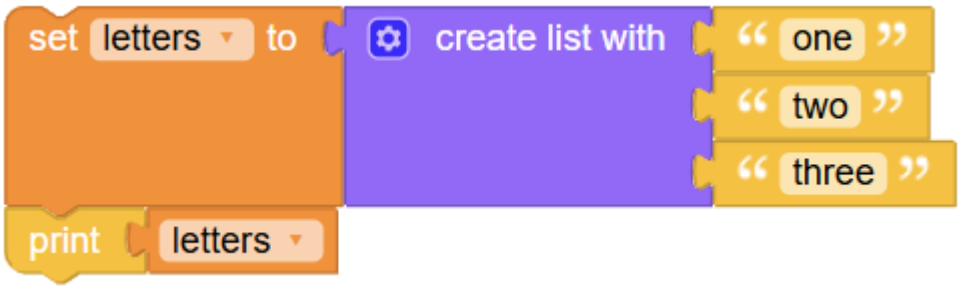
The new text returned in the example above is **"311-555-2368"**.

Related Blocks

Print List

The Text "Print" block can be used to print a list.

The result of the program below is displayed in an alert dialog:



JavaScript Code

```
var letters;

letters = ['one', 'two', 'three'];
window.alert(letters); // Print the list
```

Python Code

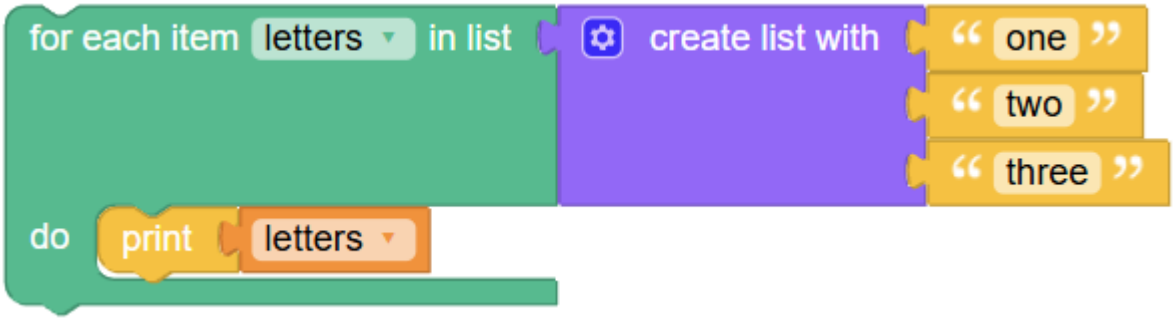
```
letters = None

letters = ['one', 'two', 'three']
print(letters)  # Print the list
```

`one, two, three` is printed.

For Each Item in a List

The "for each item" block in Loops performs an operation on each item in a list.

For example, the blocks below print each item in the list individually:



JavaScript Code

```
var letter;

var letters_list = ['one', 'two', 'three'];
for (var letters_index in letters_list) { // Print each item in the list
    letter = letters_list[letters_index];
    window.alert(letter);
}
```

Python Code

```
letter = None
for letter in ['one', 'two', 'three']:  # Print each item in the list
```

```
    print(letter)
```

one, two, and three are printed sequentially.

This operation does not remove items from the original list.

You can also refer to examples of the

Loop Termination Block.

List Order

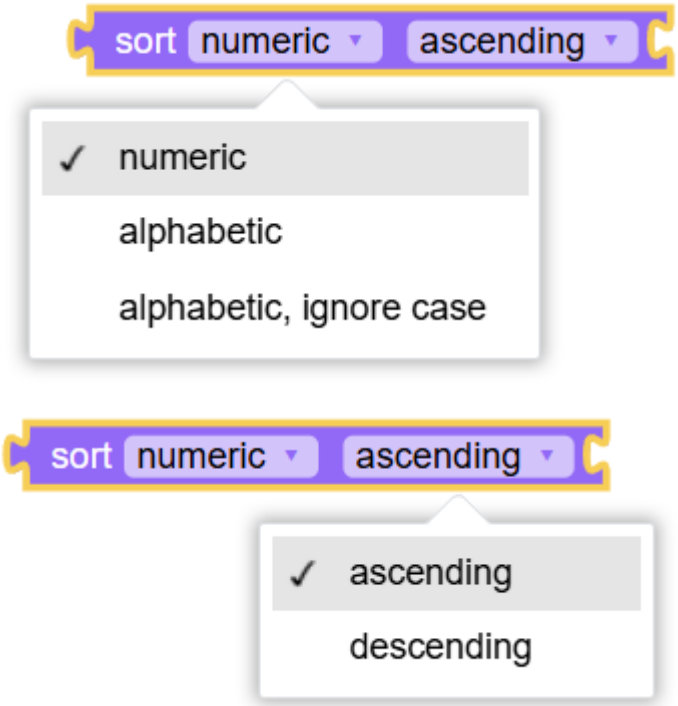
These blocks allow you to change the order of items in a list.

Sort

This block **sorts a list** based on the selected criteria.

You can sort by numbers or alphabetic order, and choose either ascending or descending order.

Additionally, when sorting alphabetically, you can choose to ignore case sensitivity.



JavaScript Code

```
function listsGetSortCompare(type, direction) { // Text comparison function
    var compareFuncs = {
```

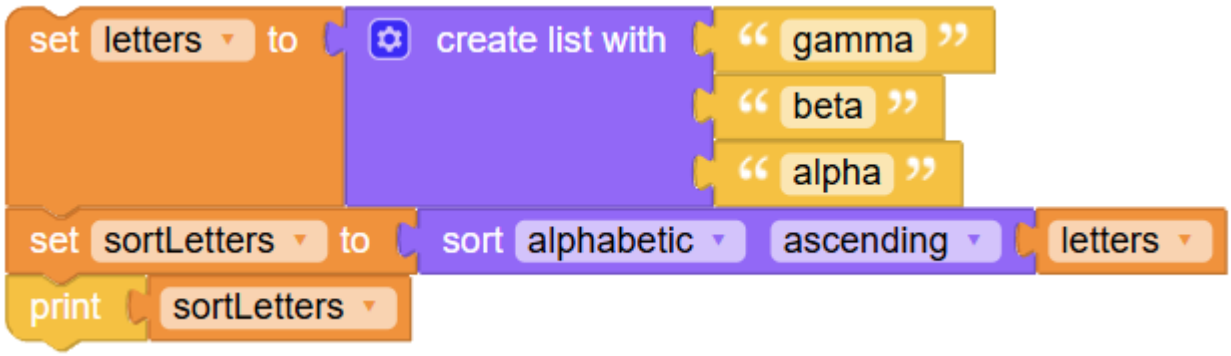
```
        'NUMERIC': function(a, b) {
                return Number(a) - Number(b); },
        'TEXT': function(a, b) {
                return String(a) > String(b) ? 1 : -1; },
        'IGNORE_CASE': function(a, b) {
                return String(a).toLowerCase() > String(b).toLowerCase() ? 1 : -1;
                },
    };
    var compare = compareFuncs[type];
    return function(a, b) { return compare(a, b) * direction; };
}

[].slice().sort(listsGetSortCompare("NUMERIC", 1));   // Numeric, ascending order
[].slice().sort(listsGetSortCompare("NUMERIC", -1));  // Numeric, descending
order
[].slice().sort(listsGetSortCompare("TEXT", 1));      // Text, ascending order
[].slice().sort(listsGetSortCompare("TEXT", -1));     // Text, descending order
[].slice().sort(listsGetSortCompare("IGNORE_CASE", 1));  // Alphabetical
(case-insensitive), ascending
[].slice().sort(listsGetSortCompare("IGNORE_CASE", -1)); // Alphabetical
(case-insensitive), descending
```

Python Code

```
def lists_sort(my_list, type, reverse):  # Sorting function
    def try_float(s):
        try:
            return float(s)
        except:
            return 0

    key_funcs = {
        "NUMERIC": try_float,                 # Numeric sorting
        "TEXT": str,                          # Text sorting
```

```
        "IGNORE_CASE": lambda s: str(s).lower()  # Case-insensitive alphabetical
        sorting
    }

    key_func = key_funcs[type]
    list_cpy = list(my_list)
    return sorted(list_cpy, key=key_func, reverse=reverse)

lists_sort([], "NUMERIC", False)      # Numeric, ascending order
lists_sort([], "NUMERIC", True)       # Numeric, descending order
lists_sort([], "TEXT", False)         # Text, ascending order
lists_sort([], "TEXT", True)          # Text, descending order
lists_sort([], "IGNORE_CASE", False)  # Alphabetical (case-insensitive),
ascending order
lists_sort([], "IGNORE_CASE", True)   # Alphabetical (case-insensitive),
descending order
```

In the example below, the list `[gamma, beta, alpha]` is sorted in **ascending alphabetical order** to create a new list called `sortLetters`.



The output result is: `[alpha, beta, gamma]`

JavaScript Code

```
var letters, sortLetters;

function listsGetSortCompare(type, direction) { // character comparison function
    var compareFuncs = {
```

```
        'NUMERIC': function(a, b) {
                return Number(a) - Number(b); },
        'TEXT': function(a, b) {
                return String(a) > String(b) ? 1 : -1; },
        'IGNORE_CASE': function(a, b) {
                return String(a).toLowerCase() > String(b).toLowerCase() ? 1 : -1;
                },
    };
    var compare = compareFuncs[type];
    return function(a, b) { return compare(a, b) * direction; };
}

letters = ['gamma', 'beta', 'alpha'];
sortLetters = letters.slice().sort(listsGetSortCompare("TEXT", 1)); // sort in
ascending alphabetical order
window.alert(sortLetters); // ['alpha', 'beta', 'gamma']
```

Python Code

```
letters = None
sortLetters = None

def lists_sort(my_list, type, reverse):  # text sorting function
    def try_float(s):
        try:
            return float(s)
        except:
            return 0
    key_funcs = {
        "NUMERIC": try_float,
        "TEXT": str,
        "IGNORE_CASE": lambda s: str(s).lower()
    }
    key_func = key_funcs[type]
```

```
    list_cpy = list(my_list)
    return sorted(list_cpy, key=key_func, reverse=reverse)

letters = ['gamma', 'beta', 'alpha']
sortLetters = lists_sort(letters, "TEXT", False)  # sort in ascending
alphabetical order
print(sortLetters)  # ['alpha', 'beta', 'gamma']
```

Reverse

This block **reverses the order of elements** in a list.

.

JavaScript Code

```
[].slice().reverse(); // Reverse the list order
```

Python Code

```
list(reversed([]))  # Reverse the list order
```

In the example below, the sorted `sortLetters` list is reversed to create the list `[gamma, beta, alpha]`.

JavaScript Code

```
var letters, sortLetters;

function listsGetSortCompare(type, direction) { // Text comparison function
    var compareFuncs = {
```

```
        'NUMERIC': function(a, b) {
            return Number(a) - Number(b);
        },
        'TEXT': function(a, b) {
            return String(a) > String(b) ? 1 : -1;
        },
        'IGNORE_CASE': function(a, b) {
            return String(a).toLowerCase() > String(b).toLowerCase() ? 1 : -1;
        },
    };
    var compare = compareFuncs[type];
    return function(a, b) { return compare(a, b) * direction; };
}

letters = ['gamma', 'beta', 'alpha'];
sortLetters = letters.slice().sort(listsGetSortCompare("TEXT", 1));
// Sort in ascending alphabetical order → ['alpha', 'beta', 'gamma']

sortLetters.slice().reverse();
// Reverse the list → ['gamma', 'beta', 'alpha']
```

Python Code

```
letters = None
sortLetters = None

def lists_sort(my_list, type, reverse):  # Text sorting function
    def try_float(s):
        try:
            return float(s)
        except:
            return 0

    key_funcs = {
```

```
        "NUMERIC": try_float,
        "TEXT": str,
        "IGNORE_CASE": lambda s: str(s).lower()
    }

    key_func = key_funcs[type]
    list_cpy = list(my_list)
    return sorted(list_cpy, key=key_func, reverse=reverse)

letters = ['gamma', 'beta', 'alpha']
sortLetters = lists_sort(letters, "TEXT", False)
# Sort in ascending alphabetical order → ['alpha', 'beta', 'gamma']

list(reversed(sortLetters))
# Reverse the list → ['gamma', 'beta', 'alpha']
```

Color

Colors are used in a wide range of graphic programs.

Blocks

Selecting a Color from the Palette

The simplest way to obtain a color is by using the **Color Picker**.
This block is displayed as a white rounded rectangle.

When you click it, a color palette appears, allowing you to select the desired color.

JavaScript Code

```
[0, 0, 0]; // Default color: Black
[255, 0, 0]; // Default color: Red
[255, 255, 0]; // Default color: Yellow
[0, 255, 0]; // Default color: Green
[0, 255, 255]; // Default color: Cyan
[0, 0, 255]; // Default color: Blue
[255, 0, 255]; // Default color: Magenta
[255, 255, 255]; // Default color: White
```

Python Code

```
[0, 0, 0] # Default color: Black
[255, 0, 0] # Default color: Red
[255, 255, 0] # Default color: Yellow
[0, 255, 0] # Default color: Green
[0, 255, 255] # Default color: Cyan
[0, 0, 255] # Default color: Blue
[255, 0, 255] # Default color: Magenta
[255, 255, 255] # Default color: White
```

Creating a Color Using Red, Green, and Blue (RGB) Values

By using the **Color Settings** block, you can specify the desired proportions of red, green, and blue. In the example below, red and blue are set to their maximum values and green is set to 0, creating a purple color.

Each color component ranges from 0 to 255 (inclusive).

JavaScript Code

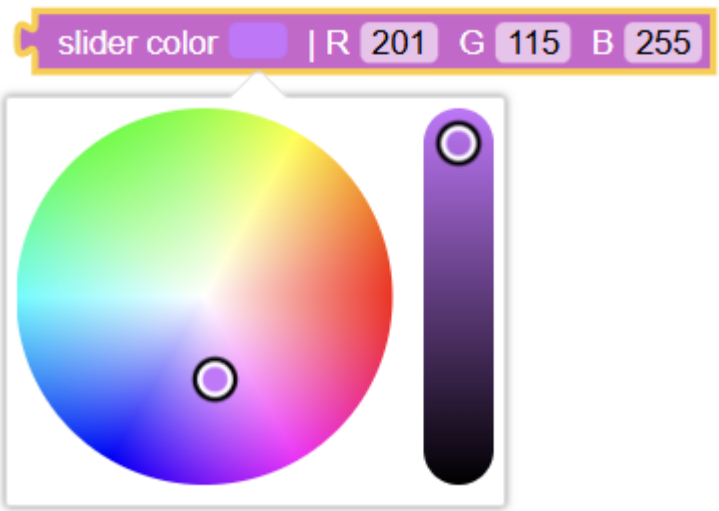
```
[255, 0, 255]; // R : 255, G : 0, B : 255
```

Python Code

```
[255, 0, 255] # R : 255, G : 0, B : 255
```

Creating Colors Using Sliders

This block provides a feature for selecting colors using **sliders**.
Users can adjust the sliders to manually combine their desired **color**.
Each slider controls the Red (R), Green (G), and Blue (B) values, and the button on the right can be used to adjust the **brightness** (lightness).



When the slider values are changed, the result is applied immediately, and the selected color is displayed in real time in the R, G, and B fields.

JavaScript Code

```
[90, 85, 224]; // R : 90, G : 85, B : 224
```

Python Code

```
[90, 85, 224] # R : 90, G : 85, B : 224
```

Generating Random Colors

The **Random Color** block generates a random color each time it is called.

This block assigns random values between 0 and 255 (inclusive) to the red, green, and blue components.

JavaScript Code

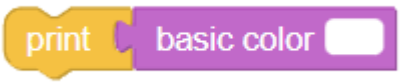
```
__randomColor(); // Generate a random color
```

Python Code

```
__randomColor() # Generate a random color
```

Technical Details

In Block Composer, colors are represented as text in the format `"rr,gg,bb"`.
Here, `"rr"`, `"gg"`, and `"bb"` represent the red, green, and blue values (0–255), respectively.
Normally, users do not see this format directly, but it can be observed by running the program below.



The program above outputs `"255,255,255"`.

JavaScript Code

```
window.alert([255, 255, 255]); // Outputs "255,255,255"
```

Python Code

```
print([255, 255, 255]) # Outputs "255,255,255"
```

As a side note, mixing light is different from mixing paint.
When red, green, and blue light are mixed in equal proportions, the result is white,
but mixing paints produces a dull, muddy color.

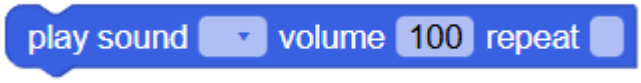
Audio

By using sound blocks, you can play various sound effects and voice audio.

Blocks

Playing Sounds

The **Play Sound** block plays a selected sound at a specified **volume**. - **Volume Control:** Allows you to adjust the sound volume. - **Loop Playback:** When the loop checkbox is enabled, the selected sound will repeat continuously.



Javascript Code

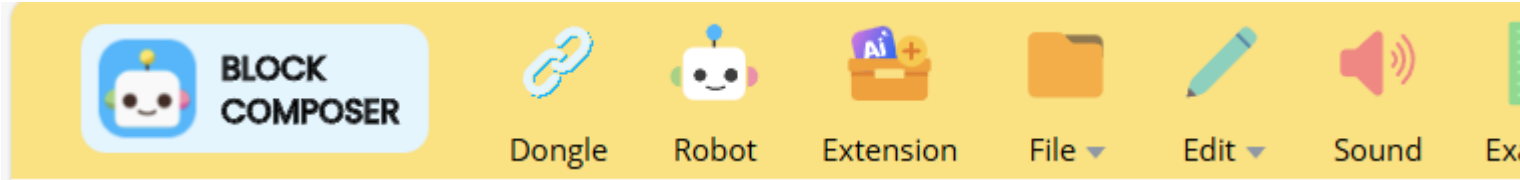
```
__playSound('', 100, false);
```

Python Code

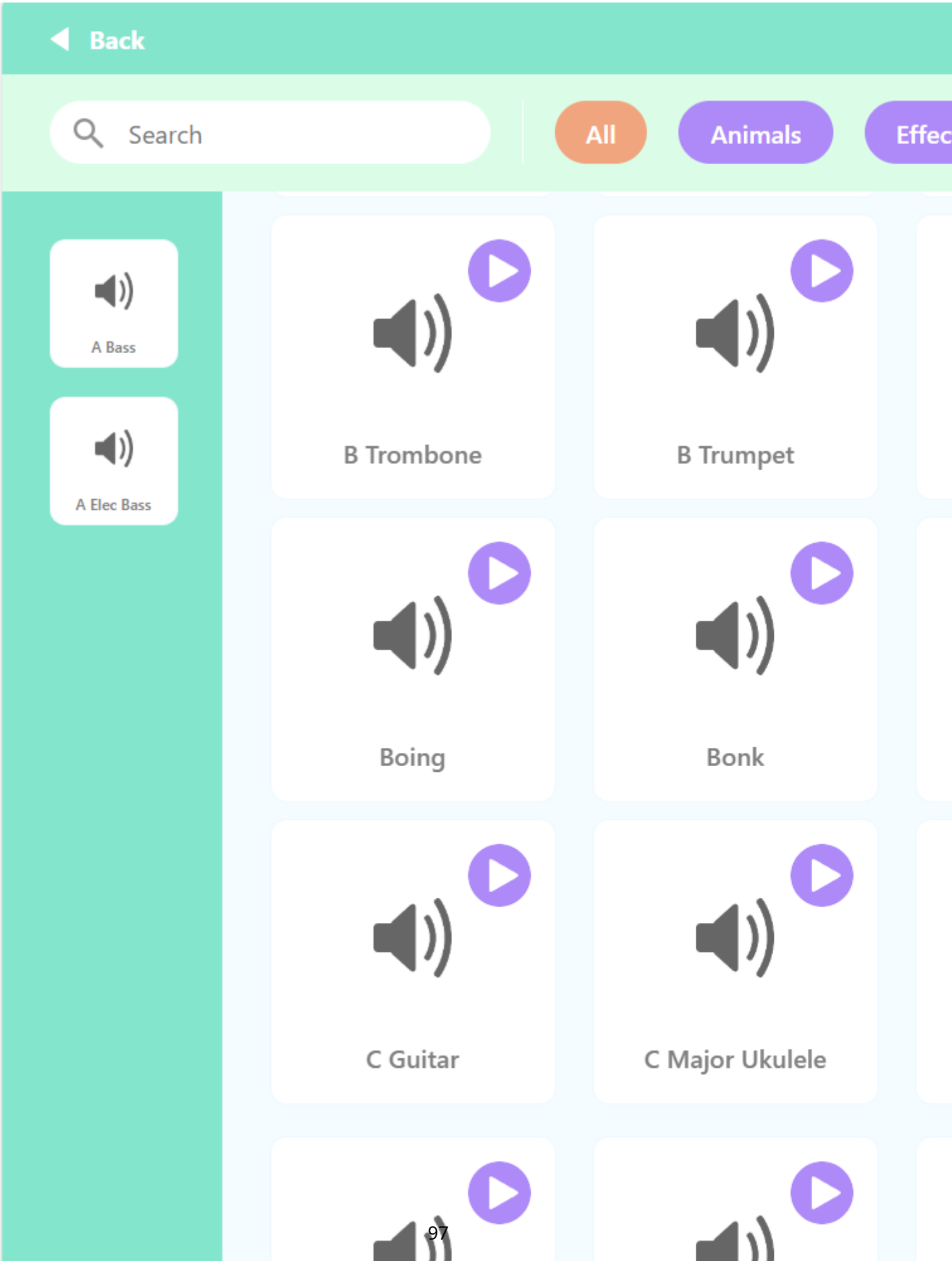
```
__playSound('', 100, False)
```

Adding Sounds

You can select the desired sound from the **Sound Menu** at the top.



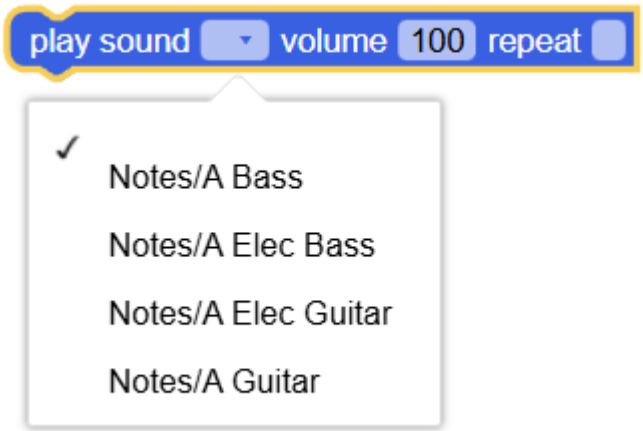
Before selecting a sound from the list, you can preview it by listening directly.


Back
Search
All
Animals
Effec
A Bass
A Elec Bass
B Trombone
B Trumpet
Boing
Bonk
C Guitar
C Major Ukulele

By using the sound adding feature, you can directly add the sound effects you want to your project. - Added sounds can be found in the list on the left. - You can preview a selected sound from the list, and delete any sounds you no longer need.

Selecting a Sound

Click the arrow below the block to view and select from the added sounds.



Javascript Code

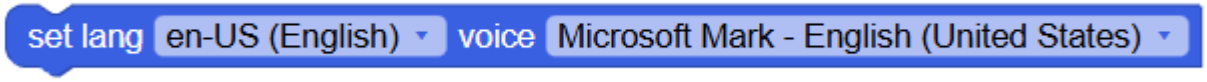
```
__playSound('Notes/A Elec Bass', 100, false); // Select the Notes/A Elec Bass sound
```

Python Code

```
__playSound('Notes/A Elec Bass', 100, False) # Select the Notes/A Elec Bass sound
```

Setting Language and Voice

This block allows you to set the **language** and **voice** used for sound playback.
You can choose from a variety of languages and voices to produce more natural-sounding speech.



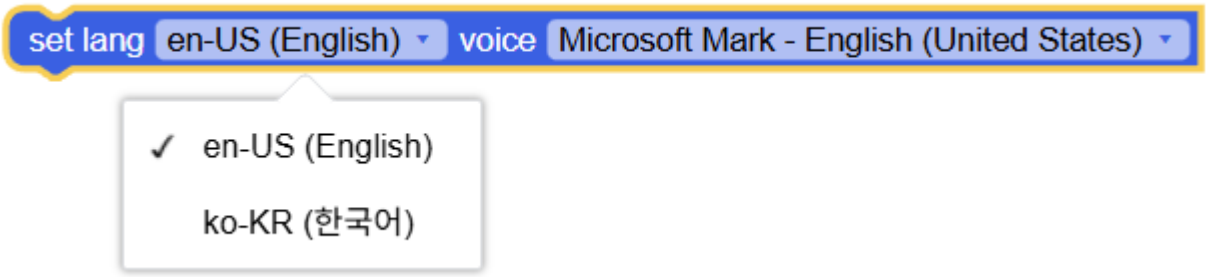
JavaScript Code

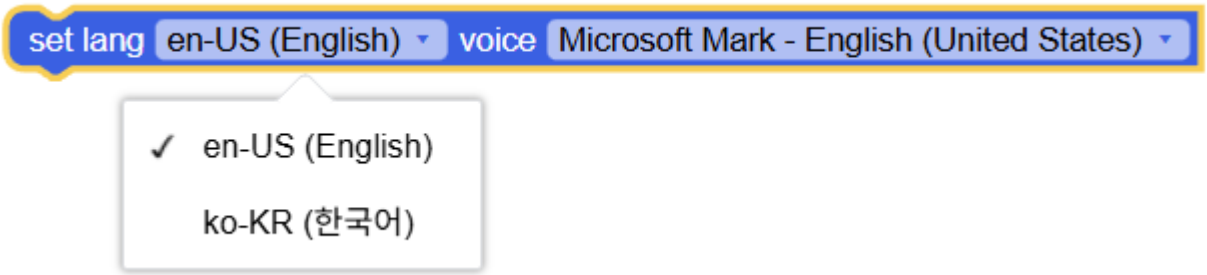
```
__setTTSOption('en-US', 'Microsoft Mark - English (United States)'); // Use English, Microsoft Mark voice
```

Python Code

```
__setTTSOption('en-US', 'Microsoft Mark - English (United States)') # Use English, Microsoft Mark voice
```

- Language Selection: You can choose the desired language.
- Voice Selection: You can choose the desired voice from a variety of options.





JavaScript Code

```
__setTTSOption('ko-KR', 'Microsoft Heami - Korean (Korean)'); // Use Korean, Heami voice
```

Python Code

```
__setTTSOption('ko-KR', 'Microsoft Heami - Korean (Korean)') # Use Korean, Heami voice
```

Speaking Text

This block converts the input text into speech and plays it aloud.
By using this block, you can output any desired **sentence as speech** in your project.

- Text Input: Enter the desired sentence to convert it into speech and play it.

JavaScript Code

```
__speak(''); // Output the sentence as speech
```

Python Code

```
__speak('') # Output the sentence as speech
```

Control

In block coding, **control blocks** are used to manipulate the flow of a program.
They can perform functions such as waiting for a certain amount of time, detecting keyboard input, and outputting logs.

Blocks

Wait

The **Wait** block executes the previous command and then **waits for a specified amount of time** before executing the next command.
By using this block, you can execute specific actions at fixed intervals or introduce a time delay between actions.

When this block is executed, it pauses for **x** seconds and then executes the next command.

Javascript Code

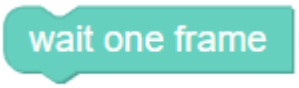
```
await __wait(x * 1000);
```

Python Code

```
import asyncio

await asyncio.sleep(x)
```

Wait 1 Frame

The **Wait 1 Frame** block pauses the program execution for one frame (approximately 0.001 seconds) and then executes the next command.
It is useful when controlling a program on a frame-by-frame basis.



When this block is executed, it pauses for one frame (approximately 0.001 seconds) and then executes the next command.

Javascript Code

```
await __wait(1);
```

Python Code

```
import asyncio

await asyncio.sleep(0.001)
```

Key Down / Key Up

- **Key Down**: Executes an action **when a specific key is pressed**
- **Key Up**: Executes an action **when a specific key is released**

The **Key Down** and **Key Up** code is shown below.

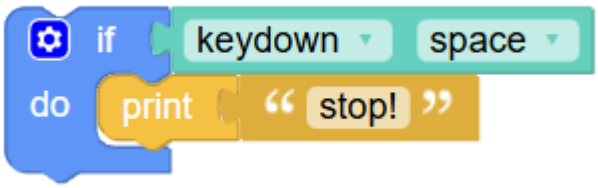
Javascript Code

```
__keydown(32); // Spacebar key down
__keyup(32); // Spacebar key up
```

Python Code

```
__keydown(32) # Spacebar key down
__keyup(32) # Spacebar key up
```

This block prints “`stop`” when the spacebar is pressed.



Javascript Code

```
if (__keydown(32)) {
    window.alert('stop!');
}
```

Python Code

```
if __keydown(32):
    print('stop!')
```

Key Input

The **Key Input** block can be configured to perform an action **when multiple keys are pressed simultaneously**.

Using this block, you can detect up to five keys at the same time, including combination keys such as `shift`, `ctrl`, and `alt`.

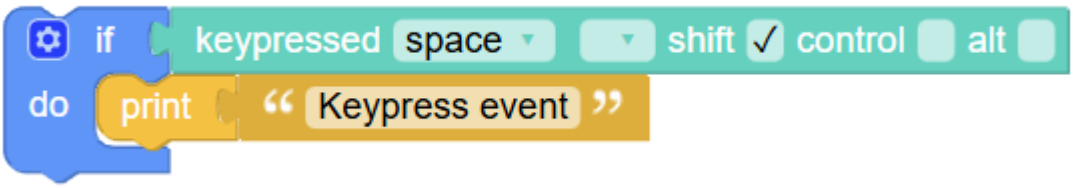
Javascript Code

```
__keypressed([32, 38, 0, 0, 0]) // When 32 (spacebar) and 38 (up arrow key) are pressed simultaneously
```

Python Code

```
__keypressed([32, 38, 0, 0, 0]) # When 32 (spacebar) and 38 (up arrow key) are pressed simultaneously
```

This block prints “Keypress event” when the Shift + Space keys are pressed.



Javascript Code

```
if (__keypressed([32, 0, 16, 0, 0])) { // When Shift + Space keys are pressed
    window.alert('Keypress event');
```

```
}
```

Python Code

```
if __keypressed([32, 0, 16, 0, 0]): # When Shift + Space keys are pressed
    print('Keypress event')
```

Log Output

The **Log Output** block displays specific variable or property values in real time in the **console**, allowing you to analyze program behavior.
It is useful for checking values or tracking data changes during the debugging process.

Javascript Code

```
__log(1, 'A', 'K') // Log output of value 1 with unit K for tag 'A'
```

Python Code

```
__log(1, 'A', 'K') # Log output of value 1 with unit K for tag 'A'
```

This is an example of checking a random integer in the console.

The result is as follows.

Log Scope

```
[A] 60K
[A] 76K
[A] 20K
[A] 96K
[A] 42K
[A] 33K
[A] 67K
[A] 7K
[A] 61K
[A] 10K
[A] 53K
[A] 96K
[A] 69K
[A] 93K
[A] 42K
[A] 39K
[A] 59K
[A] 54K
```

Javascript Code

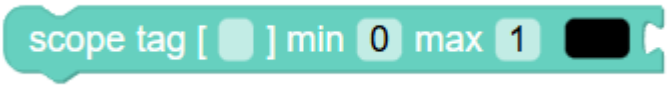
```javascript
var random;

function mathRandomInt(a, b) { // Random number generation function
    if (a > b) {
        var c = a;
        a = b;
        b = c;
    }
    return Math.floor(Math.random() * (b - a + 1) + a);
}

random = mathRandomInt(1, 100);
__log(random, 'A', 'K'); // Log a random integer between 1 and 100 with tag 'A' and unit 'K'
```

Python Code

```
import random

random2 = None
random2 = random.randint(1, 100)
__log(random2, 'A', 'K') # Log a random integer between 1 and 100 with tag 'A'
and unit 'K'
```

Scope Output

The **Scope Output** block displays changes in specific values as a real-time graph in the **scope** window, allowing intuitive data analysis.
You can configure the graph's color, minimum/maximum values, and range, making it useful for **data visualization** from sensors or for tracking variable changes.



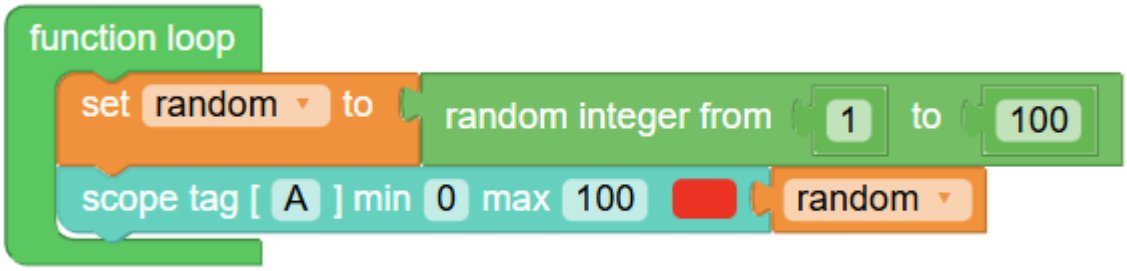
Javascript Code

```
__scope('_', 0, 1, '#000000', 0); // Display a black scope for value 0 within the
range 0 ~ 1
```

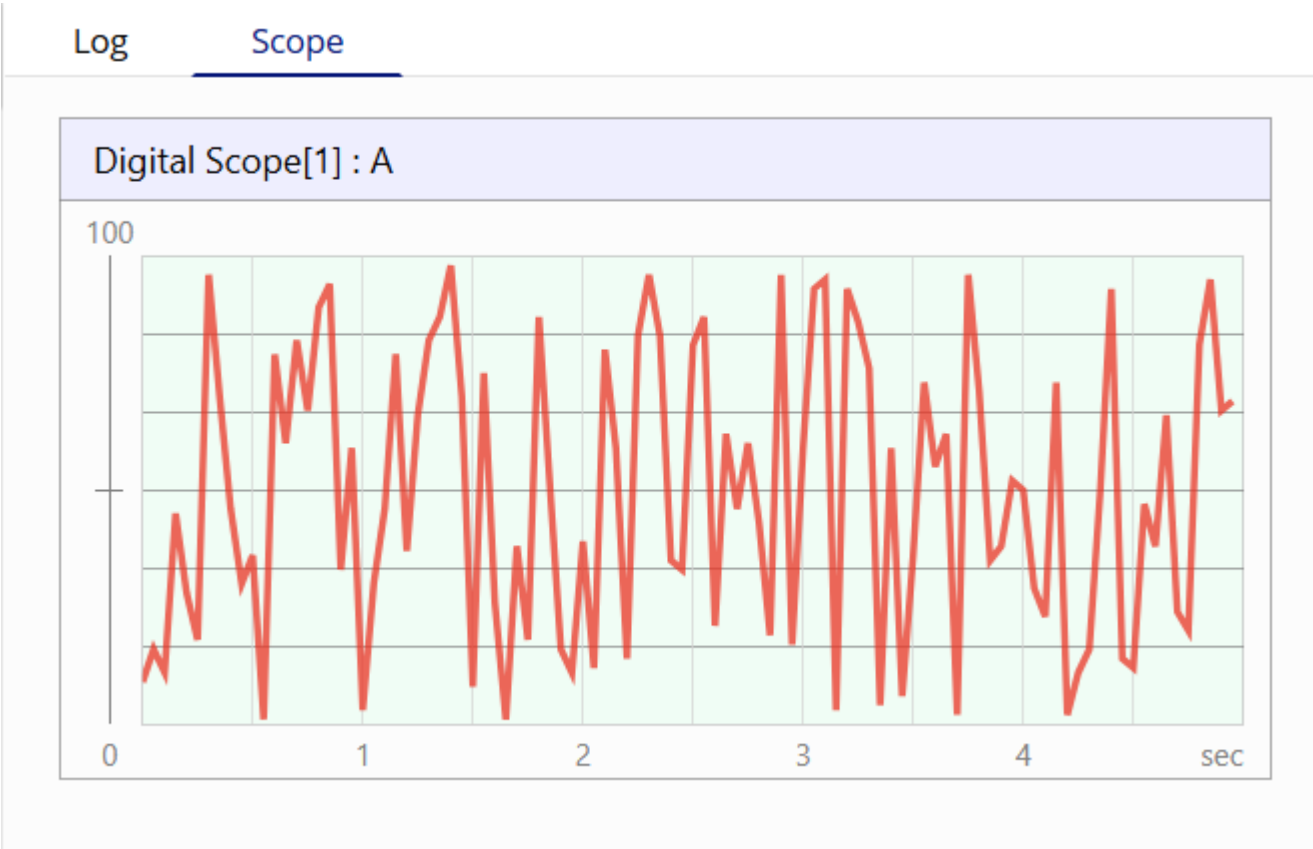
Python Code

```
__scope('_', 0, 1, '#000000', 0) # Display a black scope for value 0 within the
range 0 ~ 1
```

This is an example of checking a random variable value using a red graph.



The result is as follows.



Javascript Code

```
var random;

function mathRandomInt(a, b) { // Random number generation function
    if (a > b) {
        var c = a;
        a = b;
        b = c;
```

```
    }
    return Math.floor(Math.random() * (b - a + 1) + a);
}

random = mathRandomInt(1, 100);
__scope('A', 0, 100, '#ff0000', random); // Display a red scope with tag 'A' for
a random integer between 1 and 100 within the range 0 ~ 100
```

Python Code

```
import random

random2 = None
random2 = random.randint(1, 100)
__scope('A', 0, 100, '#ff0000', 0) # Display a red scope with tag 'A' for a
random2 integer between 1 and 100 within the range 0 ~ 100
```

Variables

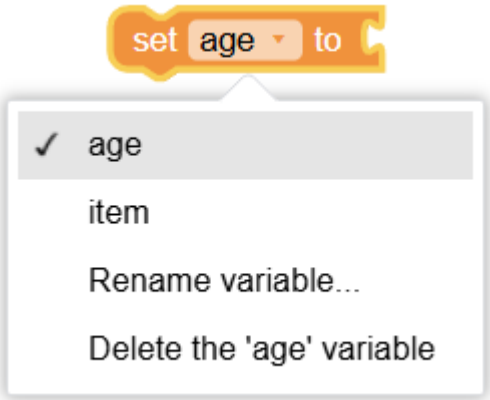
We use the term **variable** with the same meaning as in mathematics and other programming languages. That is, a variable is a **named element that can store a value and whose value can be changed**.

Variables can be created in several different ways:

- Some blocks, such as **Count with** and **For Each Item**, use variables and define their values. These variables are traditionally called **loop variables** in computer science.
- **User-defined functions** can define input values, which create variables (parameters or arguments) that are only available inside the function.
- Users can create variables at any time using the **set** block. These are traditionally called **global variables**.
- The Block Composer does **not** support **local variables**.

Dropdown Menu

When you click the dropdown arrow (triangle) next to a variable, the following menu appears.



From this menu, you can select the following options:

- All existing variable names defined in the program are displayed.
- **"Rename variable"**: Renames the selected variable throughout the entire program. When this option is selected, a dialog appears where you can enter a new name.
- **"Delete variable"**: Deletes all blocks in the program that reference this variable.

Blocks

Set

The **Set** block assigns a value to a variable.

If the variable does not already exist, a new variable is created.

For example, the block below creates a variable named "age" and assigns it the value 12.

JavaScript Code

```
var age;
age = 12; // Assigns the value 12 to the variable age
```

Python Code

```
age = None
age = 12 # Assigns the value 12 to the variable age
```

Get Value

The block below retrieves the value stored in a variable, but does not modify that value.

It is possible to use a variable without a **Set** block, but this is **not considered good programming practice**.

JavaScript Code

```
age; // Get the value of the variable
```

Python Code

```
age # Get the value of the variable
```

Change

The **Change** block adds a number to the current value of a variable.

The block above is a shorthand expression that performs the same operation as the following code.

JavaScript Code

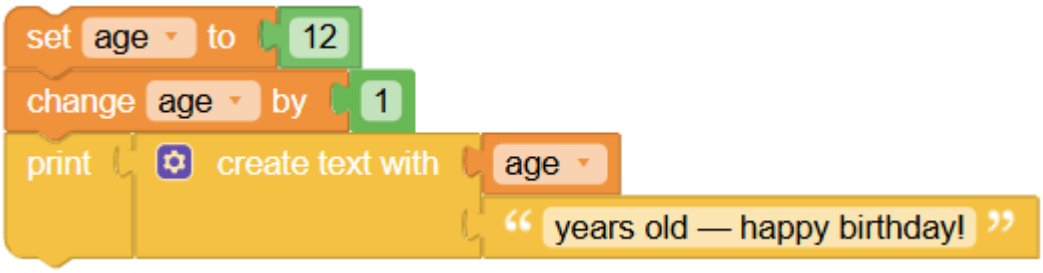
```
var age;
age = (typeof age === 'number' ? age : 0) + 1; // Increase the value of age by 1
```

Python Code

```
from numbers import Number

age = None
age = (age if isinstance(age, Number) else 0) + 1 # Increase the value of age by 1
```

Example

The following is an example demonstrating how variables are used.



The first block on the first line creates a variable named "age" and sets its initial value to the number 12.

The second block retrieves the value 12, adds 1 to it, and then stores the result (13) back into the variable.

The final line displays the following message: “13 years old — happy birthday!”

JavaScript Code

```
var age;

age = 12;
age = (typeof age === 'number' ? age : 0) + 1; // Add 1 to age
window.alert(String(age) + ' years old – happy birthday!');
```

Python Code

```
from numbers import Number

age = None
```

```
age = 12
age = (age if isinstance(age, Number) else 0) + 1 # Add 1 to age
print(str(age) + ' years old — happy birthday!')
```

Functions

A **Function** is a **collection of blocks (commands)** that performs a specific task.
By defining frequently used actions as a single function, you can manage code in a **concise and efficient** way.

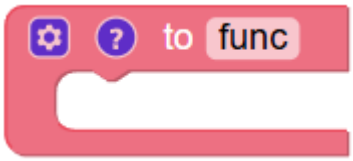
Functions have the following characteristics:
- **Reusability**: Once defined, a function can be called and used multiple times.
- **Input and Output**: A function can receive parameters (input values), process them, and can return a result (output value).
- **Improved Readability**: Functions help structure the program flow in a logical and organized manner.

Function Blocks

In block-based coding, users can define custom functionality using **function blocks**.

Function Definition Block

By using a function definition block, you can create a **new function**.
The example below shows how to define a function named `func`.



JavaScript Code

```
function func() { // define the function func
}
```

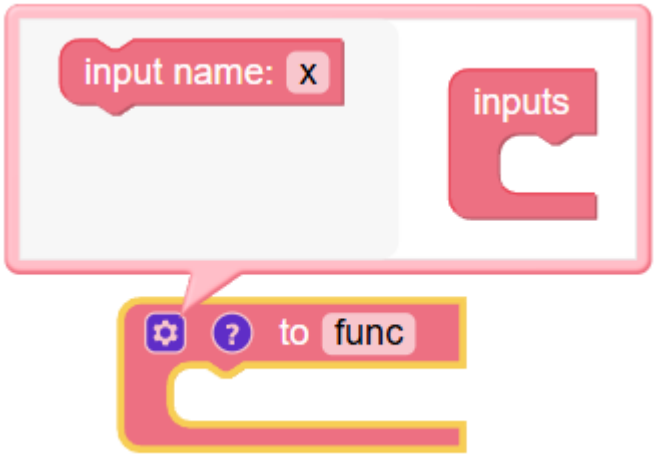
Python Code

```python
def func():  # define the function func
    pass
```

Functions with Parameters

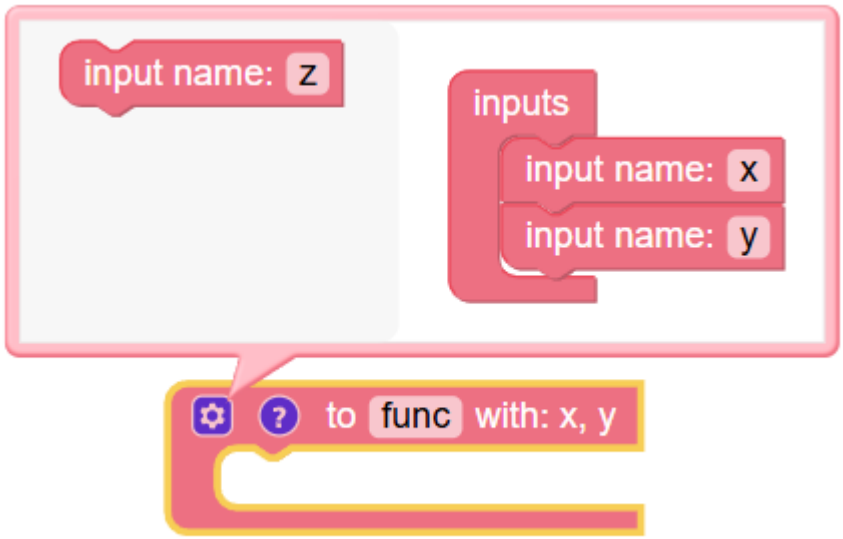
The function block has a gear button, which you can click to **add parameters (input values)**.

- **Parameters** are **input values that are passed to a function**.



How to Set Parameters

1. Add the required variables (such as x, y, etc.) in the parameter block.
2. When they are connected to the parameter list on the right, a function with parameters is created.



You can define a function `xyFunction` that has parameters x and y, and use those parameters inside

the function.

JavaScript Code

```
var x, y;

function xyFunction(x, y) { // Define xyFunction with parameters x and y
}
```

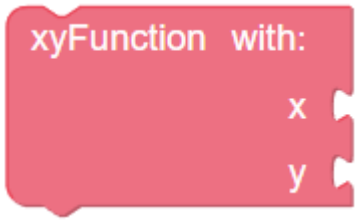
Python Code

```
x = None
y = None

def xyFunction(x, y):  # Define xyFunction with parameters x and y
    pass
```

Calling a Function (User-Defined Function Block)

When a function is defined, a **user-defined function block** is automatically created.
Using this block, you can **call and execute the predefined function**.

Below is an example of a block that calls the previously created xyFunction **function**.



JavaScript Code

```
var x, y;

function xyFunction(x, y) {
}
```

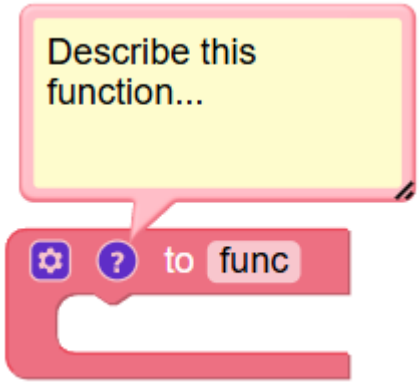
```
xyFunction(null, null); // function call
```

Python Code

```
x = None
y = None

def xyFunction(x, y):
    pass

xyFunction(None, None)  # function call
```

In addition, by clicking the ? icon, you can add a a description (comment) to the function, making it easier to explain and understand what the function does.

The image below shows an example of adding a **comment** to a function.



JavaScript Code

```
// Describe this function...
function func() {
}
```

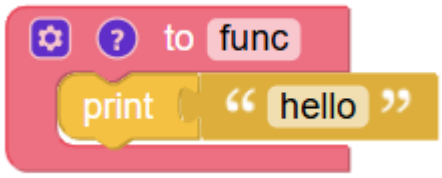
Python Code

```python
# Describe this function...
def func():
    pass
```

Functions Without a Return Value

A function without a return value performs a specific action but **does not return any value**.

The following example shows a function that prints `"hello"`.
This function only outputs `"hello"` and does not return any specific value.



JavaScript Code

```javascript
function func() { // Define a function without a return value
    window.alert('hello');
}
```

Python Code

```python
def func():  # Define a function without a return value
    print('hello')
```

When you run the block below, `"hello"` is displayed on the screen.

JavaScript Code

```
function func() {
    window.alert('hello');
}
func(); // Call a function with no return value
```

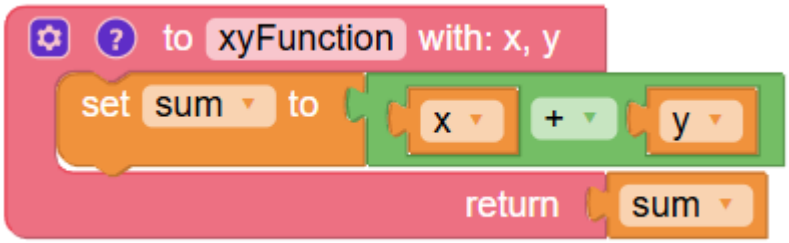
Python Code

```
def func():
    print('hello')  # Call a function with no return value
func()
```

Functions with a Return Value

A function with a return value performs a specific task and then **returns a result value** that can be used by other blocks or code.

The function below returns the sum of two numbers.
It **takes parameters x and y and returns the result of x + y**.



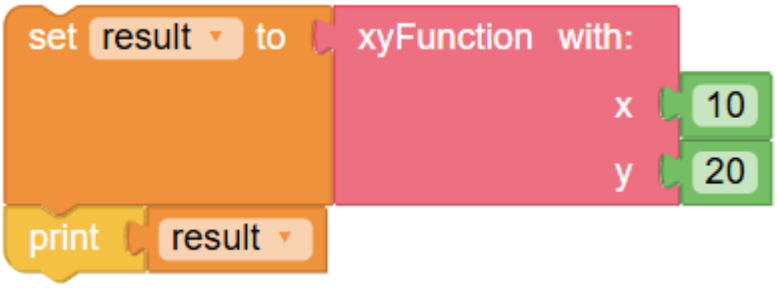
JavaScript Code

```
var x, y, sum;

function xyFunction(x, y) { // Define a function with a return value
    sum = x + y;
    return sum;
}
```

Python Code

```python
x = None
y = None
sum2 = None  # Use sum2 because 'sum' is a reserved keyword in Python

def xyFunction(x, y):  # Define a function with a return value
    global sum2
    sum2 = x + y
    return sum2
```

You can call the function below and store the returned result in the variable `result` for later use.



Function Execution Example (Returns 10 + 20 = 30)

`30` is printed as the output.

JavaScript Code

```javascript
var x, y, result, sum;

function xyFunction(x, y) {
    sum = x + y;
    return sum;
}

result = xyFunction(10, 20); // Call a function with a return value
window.alert(result);
```

Python Code

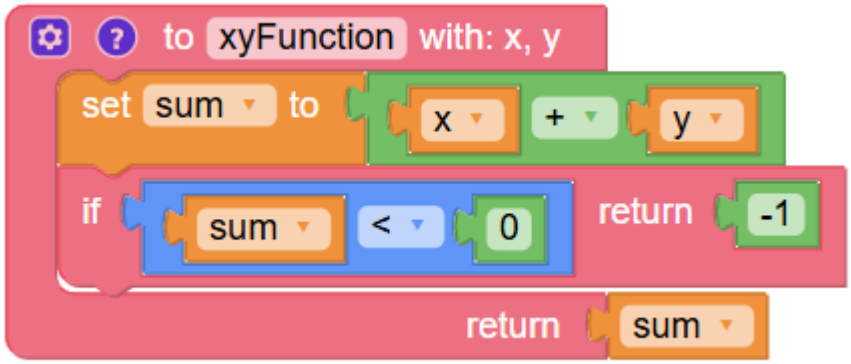
```python
x = None
y = None
result = None
sum2 = None  # Use sum2 because sum is a reserved keyword

def xyFunction(x, y):  # Function with a return value
    global result, sum2
    sum2 = x + y
    return sum2

result = xyFunction(10, 20)
print(result)
```

Block That Returns a Value Under a Specific Condition (If Return Block)

This block performs the function of **immediately returning a value and terminating the function when a specific condition is satisfied** within a function.

This block **can only be used inside a function**, and it is disabled in all other contexts.

This function returns the sum of x and y, but if the sum is less than 0, it returns -1 instead.

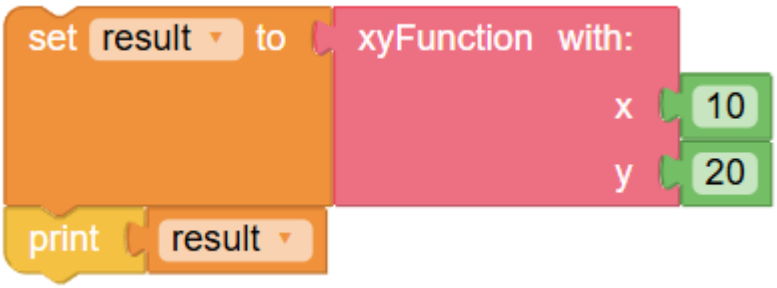


JavaScript Code

```javascript
var x, y, sum;

function xyFunction(x, y) {
    sum = x + y;
    if (sum < 0) { // If sum < 0, return -1
        return -1;
    }
    return sum;
}
```

Python Code

```python
x = None
y = None
sum2 = None  # Use sum2 because sum is a reserved keyword

def xyFunction(x, y):
    global sum2
    sum2 = x + y
    if sum2 < 0:  # If sum < 0, return -1
        return -1
    return sum2
```



-1 is printed.

JavaScript Code

```javascript
var x, y, result, sum;

function xyFunction(x, y) {
    sum = x + y;
    if (sum < 0) {
        return -1;
    }
    return sum;
}

result = xyFunction(-10, -20); // (-10) + (-20) < 0, so return -1
window.alert(result);
```

Python Code

```python
x = None
y = None
result = None
sum2 = None  # Use sum2 because sum is a reserved keyword

def xyFunction(x, y):
    global result, sum2
    sum2 = x + y
    if sum2 < 0:
        return -1
    return sum2

result = xyFunction(-10, -20)  # (-10) + (-20) < 0, so return -1
print(result)
```

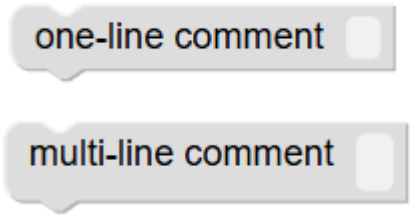
Others

In block coding, **Other Blocks** consist of blocks that do not affect program execution or robot behavior. They are used to add comments or force the termination of program execution.

Blocks

Comments

The **Comment** block allows you to add explanations without affecting code execution. without affecting code execution.
Using comments improves code readability and makes maintenance easier.



JavaScript Code

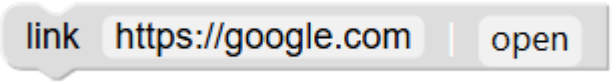
```
// Single-line comment

// Multi-line comment
// Continued input
```

Python Code

```
# Single-line comment

# Multi-line comment
# Continued input
```

Link

Using the **Link** block, you can add a **page URL** you want to open by leveraging the **comment** feature. When you click the **Open** button, it navigates to the page at the entered link.



JavaScript Code

```
// https://google.com
```

Python Code

```
# https://google.com
```

Exit

The **Exit** block immediately stops program execution and refreshes the page and resets the program to its initial state.
It is useful when you need to add a forced termination feature under specific conditions.

Javascript Code

```
__exit(); // Exit
```

Python Code

```
__exit() # Exit
```